

Learning how to Forget: Fine-tuning for Long-Context Sparse Attention

Matthias Seeger*
Amazon Web Services
mseeger@gmail.com

Zeyu Zhang
University of Amsterdam
z.zhang2@uva.nl

Vihang Patil
Amazon
pvihang@amazon.de

Konstantinos Benidis
Amazon Web Services
kbenidis@amazon.de

Sebastian Schelter
Technical University Berlin
schelter@tu-berlin.de

August 21, 2026

Abstract

A lot of prior work addressed key-value (KV) cache selection and compression by sparse attention to enable long-context inference for transformer language models without excessive hardware budgets. We provide a new method for fine-tuning models with sparse attention. It works for any KV cache policy, runs on a moderate hardware budget (e.g., a single Nvidia A100 GPU with 40 GB RAM), and allows the model to co-adapt with the policy, often outperforming models trained with exact attention (sequence parallelism). We also provide an efficient implementation of H2O sparse attention (the leading policy in our experiments) with dedicated scaled dot product attention kernel support. **KeysAndValues**, a new open source library for long-context inference and fine-tuning, provides easy-to-use and performant code for all methods discussed here.

1 Introduction

Modern large language models need to process very long contexts (i.e., number of tokens) for calling many tools with sizable outputs [47, 16], running chain of thought reasoning [62], or sustaining multi-turn conversations. While naive transformer implementations scale quadratically in compute and linearly in memory with context width, a lot of progress has been made on approximations with essentially linear time and constant memory scaling. A particularly fruitful direction is *sparse attention*, where key-value (KV) information is stored in a fixed-size KV cache, slots of which are evicted once it is full, and many different eviction policies have been proposed.

In this paper, we address the problem of how to *post-train* a transformer language model with sparse attention on a moderate hardware budget (our experiments are run computing gradients for a 4B weights model on a single¹ Nvidia A100 GPU with 40 GB RAM).

*Correspondence to mseeger@gmail.com

¹We use 4 GPUs with distributed data parallel training to attain a larger batch size.

Our novel method works for any KV cache policy and requires no further approximations beyond sparse attention. As we demonstrate in experiments on a range of long-context benchmarks, our training algorithm allows the model to co-adapt with the KV cache policy, often outperforming models trained with exact attention (sequence parallelism). Moreover, our fine-tuning method runs on resources comparable to sparse attention inference. It can be combined with orthogonal KV cache compression strategies such as grouped query attention [1] or quantization [27, 40].

The heavy-hitter oracle (H2O) [72] is one of the most prominent sparse attention policies. We demonstrate several improvements to H2O, leading to a much more efficient implementation with dedicated scaled dot product attention (SDPA) kernel support. Variants of H2O outperform other KV cache policies in our experiments, and our fast implementation takes a big step towards latencies competitive with SotA inference libraries such as vLLM [30], which use context or sequence parallelism almost exclusively. In summary, our contributions are:

- New method for fine-tuning transformer language models with sparse attention and arbitrary KV cache policy in place. This method runs on resources comparable to sparse attention inference. It combines nested activation checkpointing and CPU off-loading with exploiting a linear KV cache buffer recurrence by way of autograd saved tensors packing. Our method can process sequences of arbitrary length with constant resources.
- Methodological and implementation improvements of heavy-hitter oracle (H2O) cache policy [72]. In particular, we provide *Triton* code to return summed attention weights alongside a FlashInfer SDPA kernel [68].
- A comprehensive evaluation on a range of long-context benchmarks. Our training algorithm often outperforms models trained with sequence parallelism [32] when sparse attention inference is used.
- **KeysAndValues**, a novel open source library for long-context inference and fine-tuning (https://github.com/aws-labs/keys_values).

2 Related Work

There is a large body of work on long-context inference by way of KV cache compression. A simple idea is to group heads, so that less key and value vectors need to be stored [50, 1, 6], or to impose a low-rank structure in the query-by-key matrix [14]. These are modifications to be used during pre-training already. Cache buffers can be quantized to 8 or 4 bits, or even below [27, 40, 75, 35, 52, 31, 54, 74]. Sparse attention is a powerful general idea (discussed in Section 3.1) with many instantiations. Big Bird [73] prescribes fixed attention sparsity patterns. The heavy-hitter oracle (H2O) [72] is discussed in Section 3.1.1. Q-Hitter [77] combines H2O with quantization, steering decisions by quantizability as well. SnapKV [36] uses summed attention weights like H2O, but makes decisions only at one point during generation. Expected Attention [11] tries to estimate future relevance of KV cache information (under some strong assumptions). FlexGen [51] shows how to maximize throughput by using a cache hierarchy. FastGen [20] provides a meta-strategy voting between a number of

different cache policies. CAKE [45] runs sparse inference with a H2O-related score, but also distributes an overall memory budget between layers. Other sparse attention techniques include [23, 65, 59, 7, 64, 58, 17, 67, 61]. KVPop [24] learns a cache policy against a future-attention target, computed efficiently using FlexAttention [13]. Policies are parameterized as small xLSTMs [4]. qTTT [3] uses a few gradient updates at test time in order to improve inference results. MInference [29] and KVPress (<https://github.com/NVIDIA/kvpress>) are open source libraries providing several sparse attention methods. ShadowKV [56] is a high-throughput long-context inference system including KV cache selection. SCBench [37] provides a comprehensive empirical analysis of long-context inference methods.

Optimized scaled dot product attention (SDPA) kernels are essential for fast inference and training, pioneered by FlashAttention [9, 48]. FlashInfer [68] is optimized for the inference case $1 \ll N_q \ll N_k$ (notation from Section 3.1). FlexAttention [13] allows to specify mask and score modification code, using `torch.compile` under the hood.

Our main contribution is on long-context *fine-tuning*. Prior work can be ordered into two groups. Proposals in the first group modify multi-head attention in ways which remedy the difficulties detailed in Section 3.2. LongLoRA [8] uses permutation and reshaping of keys and values, essentially trading (B, N) for $(B(N/N_C), N_C)$, where B is batch size, N is sequence length, N_C is cache length. This speeds up MHA, but does not reduce KV memory, and works only if N/N_C is small. Native sparse attention (NSA) [70] bakes a mixture of some sparse attention kernels with different fixed policies into the model architecture. DeepSeek sparse attention (DSA) [15] is a refined variant of this idea. Different to *selective* sparse attention, this needs to be used during pre-training already, whose cost is significantly increased. IndexCache [2] speeds up DSA somewhat by sharing the indexer (i.e., the cache logic) between several layers. Also, by fixing the selection policy as part of the model choice, DSA offers considerable less flexibility during post-training. Finally, LSA and DSA still require storing the complete KV cache (even though each attention call uses only a part of it). LongGen [19] uses static sparsity patterns for the top and bottom third of layers and full attention for the middle third, which speeds up training a bit, but does not reduce its memory requirements. DMC [42] uses a form of KV cache compression where new information is either appended or accumulated with the last recent slot. They propose heuristics to train a model with this mechanism in place. This approach seems restricted to KV cache updates during generation, it is not clear what is done when a large prompt needs to be processed. Starting from Mamba [22], there have been several attempts to resurrect LSTMs [26], e.g. [10, 4]. When compared against the long-context transformer SotA, none of them have been competitive enough in order to warrant costly pre-training efforts. YOCO [57] proposes an architecture different to the transformer, where a single KV cache block serves all layers. Note that KV buffer scaling with layers is not a major problem in practice, since at any time, all but one can be offloaded to CPU (see Section A.4.1).

In the second group, KV cache buffers are not compressed, but both storage and computation are distributed across several devices. This can be done with RingAttention [38] or more recent variants [39], in what is called context parallelism (CP), or with sequence parallelism (SP) [32]. While [34] observe that long sequences can be split into chunks, and that the computation graph factorizes along the chunk axis, the factor for the final chunk depends on *all* KV cache buffers of all layers, so cannot be represented on a single device² (attempts to sparsify this computation are heuristics, and experiments are done only on rather short

²In fact, RingAttention’s way to compute gradients makes use of this graph structure implicitly.

sequences of up to 16k tokens). OOMB [33] shares properties with our work, such as chunk-level processing, activation checkpointing, and efforts to compress KV cache buffers for *autograd*. While for exact KV caches, they run into the same issues as [34], their implementation supports NSA and DSA as well. Their way of hiding KV cache buffers from *autograd* requires a number of complex dedicated CUDA kernels, which need to be hardcoded for every sparse attention variant. We manage KV buffer size in *autograd* via delta encoding, which renders our implementation agnostic to the KV cache policy. Details are given in Section A.8. LongRoPE [12, 49] combines SP with a search for non-uniform RoPE and several lifting stages. Other work tuning position encoding, data mix and fine-tuning recipes, but not compressing KV cache, includes [63, 76, 18]. LongStraw [80] is a system designed for long context reinforcement learning, with a specific emphasis on sharing prompt graphs and KV caches between different roll-outs. It uses OOMB for gradient computations, but could likely be configured with our method as well. Highly optimized implementations of CP/SP are the state of the art for long-context inference, e.g. vLLM [30], SGLang [79], and fine-tuning, e.g. Nvidia NeMo RL (<https://github.com/NVIDIA-NeMo/RL>), MS-SWIFT [78]. In Section 3.3, we comment on reasons why sparse attention methods are less frequently used in practice, and how this could be changed.

3 Long-Context Fine-Tuning

In this section, we first introduce sparse attention and key-value caching, providing several improvements to the heavy-hitter oracle (H2O) cache policy [72], leading to a more efficient implementation with dedicated SDPA kernel support. Then, we detail our main contribution: a novel method to fine-tune models with sparse attention in place, using resources comparable to sparse attention inference. While in SotA CP/SP techniques, GPUs need to be used for sharding along the context, they can be used to increase throughput (i.e., larger batch sizes) or to handle larger models in our method.

3.1 Sparse Attention. Key-Value Caching

Multi-head attention (MHA) is the most important mechanism in modern transformer architectures [60]. At its core lies *scaled dot product attention (SDPA)*:

$$\mathbf{Y} = \text{SDPA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}), \quad \mathbf{Y}, \mathbf{Q} \in \mathbb{R}^{(B, H_q, N_q, d_h)}, \quad \mathbf{K}, \mathbf{V} \in \mathbb{R}^{(B, H_k, N_k, d_h)}. \quad (1)$$

Here, $\mathbf{Q}$ (queries), $\mathbf{K}$ (keys), $\mathbf{V}$ (values) are 4D arrays, B is the batch size, d_h the *per-head embedding dimension*, H_q, H_k are numbers of heads, and N_q, N_k are sequence lengths (i.e., the third dimension is mapping to token positions in the model context). The *model embedding dimension* is $d = H_q \cdot d_h$. Let us first assume that $H_q = H_k$ and drop the first two dimensions. Then:

$$\mathbf{Y} = \mathbf{M}\mathbf{V}, \quad \mathbf{M} = \text{softmax}\left(\text{mask}\left(d_h^{-1/2}\mathbf{Q}\mathbf{K}^T\right), \text{dim} = 1\right). \quad (2)$$

$\mathbf{Y}$ are weighted combinations of values $\mathbf{V}$ with *attention weights* $\mathbf{M} \in \mathbb{R}^{(B, H_q, N_q, N_k)}$, **softmax** applies $\mathbf{x} \mapsto \exp(\mathbf{x})/(\mathbf{1}^T \exp(\mathbf{x}))$ along rows. **mask** implements causal masking: $(\text{mask}(\mathbf{X}))_{i,j} = x_{i,j} - \infty \mathbf{I}_{\{P+i < t(j)\}}$, where P and $t(j)$ are token positions (see Section 3.3.1

for details). If arrays are indexed by (b, h, j, k) , SDPA operates on (j, k) in the same way for all (b, h) , computations are parallelized over batch and head positions.

Inference in transformers switches between prompt processing and token generation. Generating a token after a prompt of size N requires SDPA with $N_k = N, N_q = 1$, with keys and values of size (B, H_k, N, d_h) in GPU memory, for each of L model layers. Exact transformer inference therefore requires $\mathcal{O}(L \cdot N \cdot B H_k d_h)$ GPU memory for the full *key-value (KV) cache*. Even for moderate context lengths N of several hundred thousands, the KV cache far surpasses the model weights in size and cannot be stored in GPU memory as is.

A large amount of prior work confronts this problem (see also Section 2). In grouped query attention (GQA) [1], we set $H_q = H_k \cdot q_g$, $q_g > 1$, so that q_g heads map to the same query group, which reduces KV cache size by a factor of q_g . Most relevant to our work is *sparse attention* (or *selective KV caching*; e.g. [72]), where the KV cache is represented by fixed-size buffers *independent* of the context length of the model. Once all slots are filled, new information overwrites (or *evicts*) existing ones. Formally, the cache (for one model layer) is represented by arrays `keys, values` : (B, H_k, N_C, d_h) and `token_pos` : (B, H_k, N_C) . Here, N_C is the *cache length*, which is chosen as large as GPU memory permits. For up to N_C tokens, the cache is filled from left to right. After that, the *KV cache policy* $\pi_l(b, h, t) \in \{0, \dots, N_C - 1\}$ dictates where additional key-value information is written for token position $t \geq N_C$, batch position $b \in \{0, \dots, B - 1\}$ and head (or query group) $h \in \{0, \dots, H_k - 1\}$. Importantly, π_l can depend on b, h : a token may be in the cache for some batch positions and heads, and not for others. $t(b, h, j) = \text{token_pos}[b, h, j]$ lists the token position of what is stored in (b, h, j) . We do not require complex memory layouts and dedicated kernels for this setup, as for example PagedAttention [30] needs. In fact, (2) depends on absolute token positions only via `mask`. This is as sparse as the conventional triangular one, but depends on token positions $t(\cdot) = \text{token_pos}$, which through cache evictions becomes non-monotonic in general. Moreover, `torch.gather` and `torch.scatter` provide fast read and write access for these buffers (see Section A.1.1).

With sparse attention, we can run inference for any context width N . First, we process up to the first N_C tokens with a single SDPA call ($N_q = N_k = N_C$), this is known as *prefilling*. The remaining $N - N_C$ tokens are processed in *chunks* of size $S < N_C$, using SDPA calls with $N_q = S, N_k = N_C$ (see Section A.4.1 for details). Token generation uses $N_q = 1, N_k = N_C$. Memory requirements are independent of N . For each chunk, the policy $[\pi_l]$ is used to determine $B \cdot H_k \cdot S$ positions (b, h, j) which are overwritten by the new keys and values. While N_C is chosen as large as memory permits, the choice of S is more subtle. The larger S , the fewer chunks, and less sequential computation results in faster processing. The smaller S , the more fine-grained the cache policy is used, which can lead to better decisions (see also Section 3.3).

3.1.1 Variants of Heavy-Hitter Oracle

The key idea behind the heavy-hitter oracle (H2O) [72] is to make use of the attention weights $\mathbf{M} = [m_{i,j}]$, a by-product of SDPA (2). Dropping (b, h) for the moment, we have that $\mathbf{Y}_{i,:} = \sum_j m_{i,j} \mathbf{V}_{j,:}$ and $\sum_j m_{i,j} = 1$. $m_{i,j}$ quantifies how much values $\mathbf{V}_{j,:}$ are used to create $\mathbf{Y}_{i,:}$. The cumulative sum $\sum_i m_{i,j}$ can be used to score the usefulness of vectors $(\mathbf{K}_{j,:}, \mathbf{V}_{j,:})$ in the KV cache. Bringing (b, h) back, we define the H2O score after having

processed t tokens as

$$\phi_{\text{h2o}}^t(b, h, j) = \sum_{t(b, h, j) \leq s < t} m_{b, h, s, j}, \quad (3)$$

where $t(b, h, j) = \text{token_pos}[b, h, j]$ is the position represented at (b, h, j) right now. We sum over $s \in \{t(b, h, j), \dots, t\}$ because the slot is occupied from KV information corresponding to token position $t(b, h, j)$, which entered the cache only then. The larger $\phi_{\text{h2o}}^t(b, h, j)$, the more valuable this information has been so far. When asked to insert new content for S tokens, for each (b, h) , we overwrite these S slots j for which $\phi_{\text{h2o}}^t(b, h, j)$ is smallest.

In this paper, we modify the original H2O policy [72] (as provided by their implementation) in several ways. First, their code selects the same cache slots for each batch position b , using the score $\phi_{\text{h2o-orig}}^t(h, j) = \sum_b \phi_{\text{h2o}}^t(b, h, j)$. The rationale for this restriction is unclear, we implement H2O without it as well. Second, the cumulative H2O score seems to favour entries (b, h, j) which have been in the cache for longer, since more terms in $[0, 1]$ are summed then. We introduce the *normalized H2O score*: $\phi_{\text{h2o-norm}}^t(b, h, j) = (t - t(b, h, j))^{-1} \phi_{\text{h2o}}^t(b, h, j)$.

Despite convincing empirical results of H2O, both in [72] and Section 4, it is not widely used. This is mostly because current implementations of H2O are much slower than the state of the art. We need summed attention weights $\sum_i m_{b, h, i, j}$ for each (b, h, j) , as by-product of SDPA (2), but none of the fast SDPA kernels derived from FlashAttention [9] provide them. Current H2O implementations use naive SDPA implementations, which are much too slow in practice. Our implementation contains *Triton* code to return summed attention weights alongside a FlashInfer SDPA kernel [68]. In Section A.6, we show how FlexAttention [13] can be used to this end as well. We come back to efficiency of sparse attention in Section 3.3.

3.2 Fine-Tuning for Sparse Attention

How should we train a model which uses sparse attention with some KV cache policy such as H2O? As noted in Section 2, all prior long-context fine-tuning methods either restrict the MHA approximation to a particular form, or use exact MHA with KV cache buffers distributed across several devices (i.e., sequence or context parallelism). However, the choice of KV cache policy, which dictates how the model’s short term memory is organized, should influence how the model is best trained. Our results in Section 4 validate this hypothesis. *Sparse attention inference for a model trained with exact MHA and sequence parallelism (which is the SotA) often performs significantly worse than for a model trained with sparse attention and the desired policy in place.*

Fine-tuning for models with sparse attention is difficult, because an enormous amount of memory is required, while GPU memory is on short supply. We need to compute gradients for training loss functions on sequences of length $N \gg N_C$, which is done by (reverse mode) automatic differentiation (or error backpropagation, *autograd*). Autograd works by creating a computation graph during the forward pass, whose nodes store arrays needed during the backward pass. If the model has L layers and the chunk size is S , the training sequence is split into $1 + \lceil (N - N_C)/S \rceil$ chunks, the first (prefill) chunk of length N_C and subsequent chunks of length S . SDPA is called for each layer and chunk, creating at least one node of the size of the KV cache, so we need at least $\mathcal{O}(L \cdot S^{-1}(N - N_C) \cdot N_C \cdot \mathcal{D})$ of memory, where $\mathcal{D} = BH_k d_h$. Assuming $S = aN_C$ for some constant a , this is $\mathcal{O}(L \cdot N \cdot \mathcal{D})$: more than the *full* KV cache would need, and far beyond what is tractable.

We need several ideas in order to bring GPU memory requirements down to levels comparable to what inference needs. First, we avoid differentiation through the KV cache policy (which is often not even possible, and in general not tractable). Along the forward pass, we store all KV cache policy decisions in a *replay log*, containing for each chunk (and each layer l) the tokens processed, and the decisions $\{\pi_l(b, h, t)\}$. Later on, we use *replay caches*, which act like normal KV caches, except that eviction decisions are replayed from the log. If a cache policy is complex and expensive to compute, it needs to be run during the forward pass only.

Next, we use activation checkpointing [25]. Even with moderate context widths, this technique is routinely used for models with many layers. Gradients are computed in two passes: forward and backward. Different from autograd, there is no computation graph built during the forward pass, only the input tensors for each transformer layer are stored to CPU memory. The backward pass is split into L autograd calls, starting from the top. Head gradients are supplied from the previous layer, inputs are loaded from CPU. Computations graphs on GPU are L times smaller, while forward computations have to be run twice.

While standard activation checkpointing tackles large L , in long-context situations the context width N is the more serious problem. We cannot even keep complete inputs or head gradients for a single layer in GPU memory (see also Section A.4.1), let alone KV buffers attached to each chunk in the computation graph. We therefore use activation checkpointing twice, in a nested fashion. To this end, we partition chunks into *cells*: $\{(B, S, d)\} \rightarrow (B, kS, d)$, where $k = \lfloor \alpha N_C / S \rfloor$, and $\alpha > 0$ is a hyperparameter which defaults to $\alpha = 1$. The first (prefill) chunk becomes the first cell. As a rule of thumb, we group chunks into cells which occupy about the size of KV cache buffers. The complete computation graph can be seen as *lattice of cells*: rows are layers, columns are cells (i.e., groups of chunks) along the context. Our backward pass runs an outer loop over rows (layers), then inner loops over cells in each layer. Each inner loop starts with a (non-autograd) forward pass along the context, storing KV cache buffers (the inner loop "activations" to checkpoint) going into each cell to CPU. Next, autograd is run separately on each cell, starting from the right. Inputs to a cell (layer inputs from the bottom and KV cache buffers from the left) and head gradients from the top are read from CPU, while head gradients from the right stay in GPU memory. Gradients are accumulated. We reuse the same CPU and GPU buffers for all inner loops.³ A detailed summary of our method is given in Section A.4.2.

Even with nested activation checkpointing, the autograd calls still need too much GPU memory. Recall that a cell consists of $k = \lfloor \alpha N_C / S \rfloor$ chunks. Autograd stores KV cache buffers for each chunk, so needs at least $\mathcal{O}(k \cdot N_C \cdot \mathcal{D})$ memory, where $\mathcal{D} = BH_k d_h$. As detailed in Section 3.1, k must be sizable to allow cache policies to make good decisions. In this section, we detail the last (and maybe most important) idea, which cuts GPU memory by a factor of k , to $\mathcal{O}(N_C \cdot \mathcal{D})$ per autograd call. This is comparable to what is needed during inference alone.

Consider KV cache buffers (**keys**, **values**), (**keys'**, **values'**) for neighboring chunks. Their size is (B, H_k, N_C, d_h) , but they only differ in $S \cdot \mathcal{D}$ values, because a chunk consists of S

³It is tempting to store all KV cache checkpoints to CPU during the initial forward pass, along with layer inputs. However, this requires a lot of CPU memory, and the per-layer forward passes to obtain KV cache checkpoints are subdominant to all other computations.

tokens only. The relationship is simple:

$$\text{keys}' = \text{scatter}(\text{keys}, \text{index}, \text{key_new}), \text{ values}' = \text{scatter}(\text{values}, \text{index}, \text{value_new}).$$

Here, `key_new`, `value_new` are KV vectors for new tokens with sizes (B, H_q, S, d_h) , `index` is based on the cache policy π_l , determining which slots are overwritten, and `scatter`, `gather` are linear `torch` operators defined in Section A.1.1.

This is a linear recurrence, which is easily inverted:

$$\text{keys} = \text{scatter}(\text{keys}', \text{index}, \text{delta_key}), \quad \text{delta_key} = \text{gather}(\text{keys}, \text{index}), \quad (4)$$

and the same for `values`. Instead of storing `keys`, `values` for each chunk in the compute graph, it suffices to store⁴ `delta_key`, `delta_value`. Since memory requirements of autograd calls are dominated by the KV cache buffers, they are reduced by a factor of k .

While the linear recurrence relation between neighboring cache buffers is simple, implementing it in the context of *PyTorch autograd* is not. We use a mechanism called *autograd saved tensors hooks*⁵, originally intended to implement activation checkpointing by CPU offloading, which can be shaped to ours needs. In a nutshell, we use the *PyTorch* mechanism to store `(delta_key, delta_value)` in the autograd graph in place of `(keys, values)` (called "packing"), reconstructing the latter from the former and subsequent `(keys', values')` during the backward pass over chunks (called "unpacking"). The key difficulty is the non-selectiveness of the mechanism: it provides a `pack_hook` function called for all arrays *PyTorch autograd* decides to place into its graph. There is no way to tag tensors in the forward code, so they can be recognized as `pack_hook` arguments. Our solution is to create *annotations* alongside the forward pass code, storing `(index, delta_key)` for `keys`, `(index, delta_value)` for `values`. In a `pack_hook(x)` call, we relate x to current annotations: x matches `(index, delta_key)` (say) if `gather(x, index) = delta_key`. A match leads to `pack_hook(x)` returning a reference to `(index, delta_key)`, which is removed from the annotation list. Note that failing to match an annotation does not lead to errors, but at most to a bit more memory being used. More details are given in Section A.4.3.

3.3 Sparse Attention and Sequence Parallelism

While with context or sequence parallelism, the context width is strictly limited by the number and memory size of GPUs available, sparse attention inference can be run for any context width on moderate GPU resources. Moreover, redundancies which exist in multi-head attention, can be exploited by way of cache compression, and experimental results with H2O are in general not worse than with exact attention even if the KV cache is compressed to 20% or less [72, 77]. Even if many GPUs are available, using sparse attention allows us to increase batch size by way of distributed data parallel, or to keep more layers in GPU memory. A large number of sparse attention variants have been proposed (see Section 2). *Why is it then that sparse attention methods are hardly used in SotA inference libraries such as vLLM [30]?* The short answer is that latency is significantly higher with existing sparse attention implementations. Further comments are in Section A.7.

⁴This is a form of *delta encoding* (https://en.wikipedia.org/wiki/Delta_encoding).

⁵https://docs.pytorch.org/tutorials/intermediate/autograd_saved_tensors_hooks_tutorial.html

Some of the gap is due to less low level implementation support for sparse attention. Highly optimized SDPA kernels are vital for fast inference [9, 13, 68]. However, as noted in Section 3.1.1, existing kernels do not cater for sparse attention inference (see details in Section 3.3.1). Other reasons for the gap are more difficult to address. A long sequence is split into chunks, the first (prefill) chunk of length N_C (cache length), subsequent chunks of length S . Larger S means fewer chunks and lower latency. But KV cache policies can make useful eviction decisions only if S is much less than N_C (in our experiments in Section 4, we use $N_C = 32768$ and $S \in \{1024, 2048\}$). For the extreme choice $S = N_C$, the whole cache is overwritten by new content for every chunk, and the KV cache policy plays no role at all! Information cannot be kept in the cache beyond a chunk if we do not allow for substantial overlap.

For example, suppose we use 8 devices, each supporting a cache length N_C , and the sequence length is $N = 8N_C$. With RingAttention, each device holds N_C slots, and the sequence is processed in 8 sequential chunks. But for sparse attention, we need $1 + 7N_C/S$ chunks, which can be substantially larger. Despite sparse attention supporting a 8 times larger batch size via distributed data parallel (DDP), inference tends to still be slower than with RingAttention. In future work, we plan to improve sparse attention latency by appropriate kernel fusion. However, the sequential nature of decision making in sparse attention may be an inherent disadvantage over sequence or context parallelism, which may remain the best choice if a large hardware budget can be afforded for inference.

Apart from large hardware requirements to even work on long sequences, RingAttention requires $\mathcal{O}(L \cdot D)$ synchronizations between all devices per gradient update, while sparse attention DDP only needs a single gradient averaging reduction. While memory transfer between devices can be run in parallel with computations, this needs double buffering⁶ in RingAttention, doubling the GPU memory needed. The peer-to-peer memory transfer is more brittle than DDP used with sparse attention, and robust training code is more difficult to implement. Finally, the "waste by pre-allocation" issues which motivate the fairly complex PagedAttention [30], have a simpler solution with sparse inference: KV cache buffers are of a fixed length, there is no need to split⁷ them into pages along the sequence axis, and no custom SDPA code is needed.

3.3.1 Discussion: SDPA Kernels for Sparse Attention

Here, we list some ideas for SDPA kernel developers to better support sparse attention. First, we consider causal masking for sparse attention. In standard MHA (the "training case"), $(\text{mask}(\mathbf{X}))_{i,j} = x_{i,j} - \infty \mathbf{I}_{\{i < j\}}$. For sparse attention, KV information is stored in cache buffers in an ordering given by token positions $t(b, h, j)$, and the causal mask is given by $(b, h, i, j) \mapsto (-\infty) \mathbf{I}_{\{P+i < t(b, h, j)\}}$, where P is the number of tokens processed before the current MHA call (so the new information is for tokens $\{P, \dots, P + N_q - 1\}$). The new key-value information has been written into the cache already, so that $\{P, \dots, P + N_q - 1\}$ is part of $\{t(b, h, j)\}$ for each (b, h) .

⁶On each device, one set of **keys, values** are read for computation and transfer, another set is written to [38].

⁷If we frequently encounter batches shorter than the cache length, we can build up buffers in chunks. But most modern agentic AI applications come with long prompts anyway.

Unfortunately, none of the fast SDPA kernel codes we know of support such variants of causal masking in an implicitly⁸ defined way. In our implementation, we sort the token positions and reorder keys and values according to this index, separate for each (b, h) , after which we can use standard causal masking, where queries are right-aligned with keys and values. This needs extra computation and memory which could be saved with better SDPA kernel support. Based on our experience, the following simple extensions of fast SDPA kernel libraries could make a major difference for sparse attention:

- Return summed attention weights $\sum_i m_{b,h,i,j}$ (see Section 3.1.1), an array of size (B, H_q, N_k) , based on attention weights which are computed anyway. This allows for H2O and related scores to be computed, driving advanced KV cache policies.
- Allow for implicitly defined causal masks of the form $(b, h, i, j) \mapsto (-\infty)\mathbf{I}_{\{P+i < t(b,h,j)\}}$, where $t(\cdot)$ is an (B, H_k, N_k) integer array. While FlexAttention [13] supports custom mask patterns, they need to be written in terms of scalar index variables, so cannot have a 3D array as input to the compute graph.

4 Experiments

The key question addressed in our experiments is: *if long context inference uses sparse attention with a particular KV cache policy, how much is gained by fine-tuning the model with the same policy in place (using our novel method) over training it with state of the art libraries using sequence or context parallelism?* We run comparisons on the Helmet [69] benchmarks, with context widths of 64k and 128k, covering a range of cache policies:

- **lastrec** (lr): $\pi(b, h, t) = t\mathbf{I}_{\{t < N_C\}} + (\text{mod}(t - N_C, N_C - \beta) + \beta)\mathbf{I}_{\{t \geq N_C\}}$, where $\beta \in [0, N_C)$. Keeps the last recent $N_C - \beta$ and first β tokens in the cache. It is important to choose $\beta > 0$ as default "attention sink" [65]. Our default is $\beta = \min(16, \lceil N_C/8 \rceil)$.
- **smart_lastrec** (slr): Variant of **lastrec**, where the number β of initial tokens is chosen dependent on content (see Section A.2.1 for details). A simple version of this heuristic appeared in [23].
- **h2o** (h2o), **h2o_norm** (h2o^{no}), **h2o_orig** (h2o^{or}): Variants of H2O [72] (see Section 3.1.1 for details). **h2o_orig** is equivalent to their code released, where $\pi(b, h, t)$ does not depend on batch position b .

We train a **Qwen3-4B-Instruct-2507**⁹ model [66], using **AdamW** [41] with base learning rate 0.0005 for up to 5 epochs. We train LoRA weights only [28] (rank $r = 16$, $\alpha = 16$, on all linear blocks). We run on four Nvidia A100 40 GB devices with a per-device batch size of 2, using RoPE [55] and YaRN [44] for position encoding. Fine-tuning is done in different ways:

⁸We cannot pass an explicit mask matrix, which would be huge and defy the purpose of fast SDPA.

⁹This checkpoint has been post-trained to process long contexts. We still need to adjust the RoPE position encoding, which we do with YaRN.

- Sequence parallelism (sp): We use MS-SWIFT [78] for fine-tuning with DeepSpeed ZeRO-3 offload, FlashAttention, and Liger kernels enabled, running on four Nvidia A100 40GB GPUs. We use a per-device batch size of 2 and sequential gradient accumulation (effective batch size 8)¹⁰.
- Our method (us): We use a cache length $N_c = 32768$, batch size $S \in \{1024, 2048\}$, and chunks per cell multiplier $\alpha = 1$ for $S = 2048$, $\alpha = 0.75$ for $S = 1024$ (see also Section A.4.1). KV cache buffers are quantized to 8 bits using `torchao`. We run distributed data parallel optimization on 4 devices to obtain an effective batch size of 8. We evaluate the model on a heldout validation set every 10 gradient steps (5 for `pop_qa`), and choose the checkpoint with the lowest validation loss for testing.

	64k datasets								128k datasets							
	nq		tri_qa		hot_qa		pop_qa		nq		tri_qa		hot_qa		pop_qa	
	us	sp	us	sp	us	sp	us	sp	us	sp	us	sp	us	sp	us	sp
exact	-	50.7	-	79.8	-	60.0	-	62.7	-	50.7	-	68.7	-	46.3	-	57.0
lr _{2k}	33.5	57.2	75.3	57.5	53.3	62.7	43.7	60.5	26.0	33.0	50.8	52.3	31.0	46.3	34.0	25.0
slr _{2k}	47.3	56.5	74.5	60.8	50.0	67.3	44.0	56.7	26.0	33.7	61.2	51.8	34.0	42.0	37.7	22.2
h2o _{2k}	47.2	70.8	78.0	72.0	53.0	68.7	57.5	44.7	24.2	40.7	47.8	63.7	19.3	26.0	53.3	49.8
h2o _{2k} ^{no}	47.8	68.2	63.2	54.5	58.3	70.0	53.0	39.8	43.5	51.3	66.7	55.3	37.3	51.0	50.2	25.2
h2o _{2k} ^{or}	49.5	73.3	66.3	65.7	57.3	68.7	62.2	45.5	45.3	58.8	71.2	71.0	36.7	44.3	50.2	33.3
lr _{1k}	59.7	57.0	73.0	60.7	47.7	65.3	41.7	59.3	23.5	32.7	59.5	50.0	28.7	45.0	34.5	25.2
slr _{1k}	37.8	57.0	59.8	59.7	52.7	65.0	46.8	57.0	29.3	36.2	58.2	49.7	33.7	44.7	34.3	21.2
h2o _{1k}	62.0	72.8	79.7	72.0	55.0	68.0	59.3	45.8	23.2	41.3	51.7	58.7	25.3	24.3	51.0	50.5
h2o _{1k} ^{no}	47.5	71.3	62.3	59.3	61.7	73.0	51.0	43.5	42.7	53.3	70.3	58.2	42.3	54.7	44.1	26.8
h2o _{1k} ^{or}	49.0	72.2	75.7	68.7	57.0	66.7	55.0	47.3	44.3	60.5	72.0	74.8	31.0	41.3	51.0	30.8

Table 1: Results for long-context inference with 5 KV cache policies and chunk sizes $2048 = 2k, 1024 = 1k$ (rows). The first row *exact* is for exact inference (sequence parallelism). We show *SubEM* values on test splits for different Helmet datasets `nq`, `trivia_qa`, `hotpot_qa`, `pop_qa`, limiting sequence lengths to 64k or 128k tokens. Columns *us* are for models trained using our novel method with the same cache policy in place, columns *sp* are for models trained with sequence parallelism.

Results on 4 Helmet datasets (`nq`, `trivia_qa`, `hotpot_qa`, `pop_qa`) [69] and 10 different cache setups (5 policies, 2 chunk sizes) are provided in Table 1. Respective results for the base checkpoint (no fine-tuning) are given in Table 4 in the Appendix (referred to as *no* elsewhere). For further 6 Helmet datasets (`trec_coarse`, `nlu`, `clinc150`, `inf_qa`, `inf_mc`, `json_kv`), Table 2 provides results for 3 cache policies and chunk size $S = 1024$. Details about Helmet datasets and metrics are given in Section A.3.1.

For the datasets in Table 1, results are mixed and inconclusive: *sp* is best for `nq` and `hotpot_qa`, *no* for `trivia_qa` (fine-tuning does not help), and *us* for `pop_qa`. However, for the datasets in Table 2, *us* strongly outperforms *sp* and *no*. A closer look at generated samples (which can be up to 128 tokens) reveals a major failure mode of *sp* (see Section A.5.3): *its*

¹⁰For 64k (128k), 2 (4) devices cover the context, so we need 2 (4) sequential gradient accumulation steps.

dataset	trn	exact	slr _{1k}	h2o _{1k} ^{no}	h2o _{1k} ^{or}
trec_coarse	us	-	96.0	96.4	96.2
	sp	97.8	30.0	23.2	77.6
	no	-	28.2	19.8	36.0
nlu	us	-	90.0	87.4	79.8
	sp	90.2	28.6	32.8	74.0
	no	-	24.8	30.0	21.2
clinc150	us	-	97.4	96.8	94.0
	sp	97.6	64.2	61.6	68.0
	no	-	62.6	54.0	34.8
inf_qa	us	-	26.6	32.2	36.8
	sp	40.8	2.2	2.2	3.3
	no	-	2.5	2.9	3.4
inf_mc	us	-	40.0	42.0	54.0
	sp	66.0	25.0	29.0	39.0
	no	-	36.0	41.0	40.0
json_kv	us	-	49.0	50.0	3.0
	sp	100.0	0.0	0.0	1.0
	no	-	0.0	0.0	0.0

Table 2: Results for 6 additional Helmet datasets not featured in Table 1 (context width 128k). Inference under 3 KV cache policies (chunk size $1024 = 1k$), *exact* uses sequence parallelism (column). *trn* denotes model checkpoint being used: *us* uses our novel method with the same cache policy in place, *sp* is using sequence parallelism, *no* is the base checkpoint Qwen3-4B-Instruct-2507 (no fine-tuning). Note that metrics are different, depending on the dataset (see Table 3).

outputs are far too long and contain mostly random nonsense. For most datasets in Table 2, targets are single numerical values, and the *Accuracy* metric (see Section A.3.1) compares this to the most frequently occurring number in the output. Poor results are due to the output for *sp* often containing many numbers. In contrast, *us* learns how to stop properly and usually outputs a single number. In fact, *the same failure mode dominates outcomes in Table 1 just as well*, but the metric *SubEM* used for the 4 datasets ignores content or length of output, as long as the target string is contained in it. Finally, *sp* and *no* fail for *json_kv* as well, despite this using the *SubEM* metric. Targets are UUIDs of length 32 tokens. While variants of *us* identify them about half the time, they are hardly ever contained in the outputs of *sp* and *no*. In Section A.5.3, we quantify the failure mode in Table 7. While outputs for *us* are close in length to true targets, they are longer by large factors for *sp*, *no*, which often (but not always) span the full 128 tokens (despite true targets being much shorter). We also provide randomly chosen examples for outputs there, showcasing their nonsense content for *sp*.

The shortness of desired targets is a clear signal in the training data, expressed not only by *us*, but also by the *sp* checkpoints if exact inference is used with them (see Table 8). We should not be surprised by *sp* exhibiting such failure modes. When training with sequence parallelism (SP), each token can attend to any earlier one. This property is cut during inference, when most KV information is evicted at some point according to a logic which

SP was never aware of. Clearly, *models should be trained under the same conditions and restrictions which govern inference later on*. Our new method allows practitioners to do that even on a low budget, no matter what KV cache policy they like to use during inference.

While not our main focus here, the different KV cache policies exhibit variable performance across the different datasets. Ideally, the best policy is chosen for each task. Our results are inconclusive when it comes to ranking the different H2O variants (see Section 3.1.1). However, one concerning datapoint is the poor performance of $\text{h2o}_{1k}^{\text{or}}$ on `json_kv` in Table 2. In Table 7, we see that $R = 3.7$ and $p_{128} = 85\%$ for this logic, hinting to a similar failure mode than *sp* and *no*. At least in this case, the decision in [72] to score and evict batch dimensions together works much less well than the alternatives.

5 Conclusions

We showed how transformer language models with sparse attention can be fine-tuned on a moderate hardware budget (e.g., a single Nvidia A100 GPU with 40 GB RAM). Our method works for any KV cache selection or compression policy and allows the model to co-adapt with the policy, often outperforming models trained with exact attention (sequence parallelism). We also provide a much more efficient implementation of H2O sparse attention (the leading policy in our experiments) with dedicated scaled dot product attention (SDPA) kernel support. By simplifying KV cache structure and clarifying the requirements on SDPA, we hope to direct more attention of the fast inference community on sparse attention (see Section 3.3.1), which despite its major potential for post-training specialization via cache selection or compression policy design does not currently play a significant role in long-context inference or fine-tuning practice.

In future work, we will combine context parallelism with sparse attention. We are also considering kernel fusion ideas in order to narrow the latency gap further. An important direction will be multi-stream asynchronous implementations which allow for on-the-fly CPU offloading [71]. We believe that once the host memory of a system can be used without much synchronization overhead and less double buffering, many current difficulties with KV caching will be much diminished. Finally, we hope that `KeysAndValues` (https://github.com/aws-labs/keys_values), the open source library with which most experiments were run here (see Section A.9), will make it easier to use, compare and extend sparse attention policies, work on which so far is somewhat cluttered when it comes to implementations.

References

- [1] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, Lebrón, and F. Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4895–4901, 2023.
- [2] Y. Bai, Q. Dong, T. Jiang, X. Lv, Z. Du, A. Zeng, J. Tang, and J. Li. IndexCache: Accelerating sparse attention via cross-layer index reuse. Technical Report arXiv:2603.12201 [cs.CL], 2026.

- [3] R. Bansal, A. Zhang, R. Tiwari, L. Madaan, S. Duvvuri, F. Devvrit, D. Brandfonbrener, D. Alvarez-Melis, P. Bhargava, M. Kale, and S.e Jelassi. Let’s (not) just put things in context: Test-time training for long-context LLMs. In *Int. Conf. Learning Representations*, 2026.
- [4] M. Beck, K. Pöppel, M. Spanring, A. Auer, O. Prudnikova, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter. xLSTM: Extended long short-term memory. In Globerson et al. [21].
- [5] D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors. *Advances in Neural Information Processing Systems 38*. Curran Associates, 2025.
- [6] W. Brandon, M. Mishra, A. Nrusimha, R. Panda, and J. Ragan-Kelley. Reducing transformer key-value cache size with cross-layer attention. In Globerson et al. [21].
- [7] Z. Cai, Y. Zhang, B. Gao, Y. Liu, Y. Li, T. Liu, K. Lu, W. Xiong, Y. Dong, J. Hu, and W. Xiao. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. In *Conference on Language Modeling*, 2025.
- [8] Y. Chen, S. Qian, H. Tang, X. Lai, Z. Liu, S. Han, and J. Jia. LongLoRA: Efficient fine-tuning of long-context large language models. In *Int. Conf. Learning Representations*, 2024.
- [9] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In Oh et al. [43].
- [10] T. Dao and A. Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In Salakhutdinov et al. [46], pages 10041–10071.
- [11] A. Devoto, M. Jeblick, and S. Jégou. Expected attention: KV cache compression by estimating attention from future queries distribution. Technical Report arXiv:2510.00636 [cs.AI], 2025.
- [12] Y. Ding, L. Zhang, C. Zhang, Y. Xu, N. Shang, J. Xu, F. Yang, and M. Yang. LongRoPE: Extending LLM context window beyond 2 million tokens. In Salakhutdinov et al. [46], pages 11091–11104.
- [13] J. Dong, B. Feng, D. Guessous, Y. Liang, and H. He. FlexAttention: a programming model for generating optimized attention kernels. In *Proceedings of the 8th MLSys Conference*, pages 381–394, 2025.
- [14] DeepSeek-AI et al. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. Technical Report arXiv:2405.04434 [cs.CL], 2024.
- [15] DeepSeek-AI et al. DeepSeek-V3.2: Pushing the frontier of open large language models. Technical Report arXiv:2512.02556 [cs.CL], 2025.
- [16] J. Feng, S. Huang, X. Qu, G. Zhang, Y. Qin, B. Zhong, C. Jiang, J. Chi, and W. Zhong. ReTool: Reinforcement learning for strategic tool use in LLMs. Technical Report arXiv:2504.11536 [cs.CL], 2025.

- [17] Y. Feng, J. Lv, Y. Cao, X. Xie, and S. Zhou. Ada-KV: Optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference. In Belgrave et al. [5].
- [18] T. Gao, A. Wettig, H. Yen, and D. Chen. How to train long-context language models (effectively). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2391–2404, 2023.
- [19] S. Ge, X. Lin, Y. Zhang, J. Han, and H. Peng. A little goes a long way: Efficient long context training and inference with partial contexts. In *Int. Conf. Learning Representations*, 2025.
- [20] S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *Int. Conf. Learning Representations*, 2024.
- [21] A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors. *Advances in Neural Information Processing Systems 37*. Curran Associates, 2024.
- [22] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. Technical Report arXiv:2312.00752 [cs.LG], 2023.
- [23] C. Han, Q. Wang, H. Peng, W. Xiong, Y. Chen, H. Ji1, and S. Wang. LM-Infinite: Zero-shot extreme length generalization for large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 3991–4008, 2024.
- [24] L. Hauzenberger, N. Schmidinger, A. Hartl, D. Stap, T. Schmied, Böck, S. Klambauer, and S. Hochreiter. KVpop – key-value cache compression with predictive online pruning. Technical Report arXiv:2607.05061 [cs.LG], 2026.
- [25] J. Herrmann, O. Beaumont, L. Eyraud-Dubois, J. Hermann, A. Joly, and A. Shilova. Optimal checkpointing for heterogeneous chains: How to train deep neural networks with limited memory. Technical Report arXiv:1911.13214 [cs.LG], 2019.
- [26] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [27] C. Hooper, S. Kim, H. Mohammadzadeh, M. Mahoney, S. Shao, K. Keutzer, and A. Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In Globerson et al. [21].
- [28] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *Int. Conf. Learning Representations*, 2022.
- [29] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. Abdi, D. Li, C. Lin, Y. Yang, and L. Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In Globerson et al. [21].

- [30] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with Page-dAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [31] A. Łańcucki, K. Staniszewski, P. Nawrot, and E. Ponti. Inference-time hyper-scaling with kv cache compression. In Belgrave et al. [5].
- [32] S. Li, F. Xue, C. Baranwal, Y. Li, and Y. You. Sequence parallelism: Long sequence training from system perspective. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7376–7399, 2023.
- [33] W. Li, D. Yu, G. Luo, Y. Zhang, Y. Wu, J. Liu, Z. Gong, Z. Liao, F. Chao, and R. Ji. Out of the memory barrier: A highly memory-efficient training system for LLMs with million-token contexts. In *Int. Conf. Learning Representations*, 2026.
- [34] W. Li, Y. Zhang, G. Luo, D. Yu, and R. Ji. Training long-context LLMs efficiently via chunk-wise optimization. In *Findings of the Association for Computational Linguistics (ACL)*, pages 2691–2700, 2025.
- [35] X. Li, Z. Xing, M. Li, L. Qu, H. Zhen, Y. Yao, W. Liu, S. Pan, and M. Yuan. KVTuner: Sensitivity-aware layer-wise mixed-precision KV cache quantization for efficient and nearly lossless LLM inference. In Singh et al. [53].
- [36] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen. SnapKV: LLM knows what you are looking for before generation. In Belgrave et al. [5], pages 529–536.
- [37] Y. Li, H. Jiang, Q. Wu, X. Luo, S. Ahn, C. Zhang, A. Abdi, D. Li, J. Gao, Y. Yang, and L. Qiu. SCBench: A KV cache-centric analysis of long-context methods. In *Int. Conf. Learning Representations*, 2025.
- [38] H. Liu, M. Zaharia, and P. Abbeel. RingAttention with blockwise transformers for near-infinite context. In *Int. Conf. Learning Representations*, 2024.
- [39] Z. Liu, S. Wang, S. Cheng, Z. Zhao, K. Wang, X. Zhao, J. Demmel, and Y. You. StarTrail: Concentric ring sequence parallelism for efficient near-infinite-context transformer model training. In Belgrave et al. [5].
- [40] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, D. Chen, and X. Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In Salakhutdinov et al. [46], pages 32332–32344.
- [41] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. Technical Report arXiv:1711.05101 [cs.LG], 2017.
- [42] P. Nawrot, A. Łańcucki, M. Chochowski, D. Tarjan, and E. Ponti. Dynamic memory compression: Retrofitting LLMs for accelerated inference. In Salakhutdinov et al. [46], pages 37396–37412.
- [43] A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors. *Advances in Neural Information Processing Systems 36*. Curran Associates, 2023.

- [44] B. Peng, J. Quesnelle, H. Fan, and E. Shippole. YaRN: Efficient context window extension of large language models. In *Int. Conf. Learning Representations*, 2024.
- [45] Z. Qin, Y. Cao, M. Lin, W. Hu, S. Fan, K. Cheng, W. Lin, and J. Li. CAKE: Cascading and adaptive KV cache eviction with layer preferences. In *Int. Conf. Learning Representations*, 2025.
- [46] R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, editors. *International Conference on Machine Learning 41*. Proceedings of Machine Learning Research, 2024.
- [47] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In Oh et al. [43].
- [48] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In Globerson et al. [21], pages 68658–68685.
- [49] N. Shang, L. Zhang, S. Wang, G. Zhang, G. Lopez, F. Yang, W. Chen, and M. Yang. LongRoPE2: Near-lossless LLM context window scaling. In Singh et al. [53].
- [50] N. Shazeer. Fast transformer decoding: One write-head is all you need. Technical Report arXiv:1911.02150 [cs.NE], 2019.
- [51] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Fu, Z. Xie, B. Chen, C. Barrett, J. Gonzalez, P. Liang, C. Re, I. Stoica, and C. Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, editors, *International Conference on Machine Learning 40*, volume 202. Proceedings of Machine Learning Research, 2023.
- [52] A. Shutova, V. Malinovskii, V. Egiazarian, D. Kuznedelev, D. Mazur, S. Nikita, I. Ermakov, and D. Alistarh. Cache me if you must: Adaptive key-value quantization for large language models. In Singh et al. [53].
- [53] A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, and J. Zhu, editors. *International Conference on Machine Learning 42*. Proceedings of Machine Learning Research, 2025.
- [54] K. Staniszewski and A. Lańcucki. KV cache transform coding for compact storage in LLM inference. In *Int. Conf. Learning Representations*, 2026.
- [55] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568(C), 2024.
- [56] H. Sun, L. Chang, W. Bao, S. Zheng, N. Zheng, X. Liu, H. Dong, Y. Chi, and B. Chen. ShadowKV: KV cache in shadows for high-throughput long-context LLM inference. In Singh et al. [53], pages 57355–57373.
- [57] Y. Sun, L. Dong, Y. Zhu, S. Huang, W. Wang, S. Ma, Q. Zhang, Z. Wang, and F. Wei. You only cache once: Decoder-decoder architectures for language models. In Globerson et al. [21].

- [58] H. Tang, Y. Lin, J. Lin, Q. Han, D. Ke, S. Hong, Y. Yao, and G. Wang. RazorAttention: Efficient KV cache compression through retrieval heads. In *Int. Conf. Learning Representations*, 2025.
- [59] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han. Quest: Query-aware sparsity for efficient long-context llm inference. In Salakhutdinov et al. [46], pages 47901–47911.
- [60] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems 31*, pages 6000–6010. Curran Associates, 2018.
- [61] G. Wang, S. Upasani, C. Wu, D. Gandhi, J. Li, C. Hu, B. Li, and U. Thakker. LLMs know what to drop: Self-attention guided KV cache eviction for efficient long-context inference. In *Int. Conf. Learning Representations*, 2025.
- [62] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, W. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35*. Curran Associates, 2022.
- [63] T. Wu, Y. Zhao, and Z. Zheng. An efficient recipe for long context extension via middle-focused positional encoding. In Globerson et al. [21].
- [64] G. Xiao, J. Tang, J. Zuo, J. Guo, S. Yang, H. Tang, Y. Fu, and S. Han. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. In *Int. Conf. Learning Representations*, 2025.
- [65] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In *Int. Conf. Learning Representations*, 2024.
- [66] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu. Qwen3 technical report. Technical Report arXiv:2505.09388 [cs.CL], 2025.
- [67] S. Yang, J. Guo, H. Tang, Q. Hu, G. Xiao, J. Tang, Y. Lin, Z. Liu, Y. Lu, and S. Han. LServe: Efficient long-sequence LLM serving with unified sparse attention. In *Proceedings of the 8th MLSys Conference*, 2025.
- [68] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving. Technical Report arXiv:2501.01005 [cs.DC], 2025.
- [69] H. Yen, T. Gao, M. Hou, K. Ding, D. Fleischer, P. Izsak, M. Wasserblat, and D. Chen. Helmet: How to evaluate long-context models effectively and thoroughly. In *Int. Conf. Learning Representations*, 2025.

- [70] J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie¹, Y. Wei, L. Wang, Z. Xiao, Y. Wang, C. Ruan, M. Zhang, W. Liang, and W. Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. Technical Report arXiv:2502.11089 [cs.CL], 2025.
- [71] Z. Yuan, H. Sun, L. Sun, and Y. Ye. MegaTrain: Full precision training of 100b+ parameter large language models on a single GPU. Technical Report arXiv:2604.05091 [cs.CL], 2026.
- [72] Zhang Z., Y. Sheng, T. Zhou., T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Re, C. Barrett, Z. Wang, and B. Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In Oh et al. [43], pages 34661–34710.
- [73] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed. Big Bird: Transformers for longer sequences. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems 33*, pages 17283–17297. Curran Associates, 2020.
- [74] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate. In *Int. Conf. Learning Representations*, 2026.
- [75] T. Zhang, J. Yi, Z. Xu, and A. Shrivastava. KV cache is 1 bit per channel: Efficient large language model inference with coupled quantization. In Globerson et al. [21], pages 3304–3331.
- [76] Z. Zhang, R. Chen, S. Liu, Z. Yao, O. Ruwase, B. Chen, X. Wu, and Z. Wang. Found in the middle: How language models use long contexts better via plug-and-play positional encoding. In Globerson et al. [21], pages 60755–60775.
- [77] Z. Zhang, S.i Liu, R. Chen, B. Kailkhura, B. Chen, and A. Wang. Q-Hitter: A better token oracle for efficient LLM inference via sparse-quantized KV cache. In P. Gibbons, G. Pekhimenko, and C. De Sa, editors, *Proceedings of Machine Learning and Systems (MLSys)*, volume 6, pages 381–394, 2024.
- [78] Y. Zhao, J. Huang, J. Hu, X. Wang, Y. Mao, D. Zhang, Z. Jiang, Z. Wu, B. Ai, A. Wang, W. Zhou, and Y. Chen. SWIFT: A scalable lightweight infrastructure for fine-tuning. In *Proceedings of the 39th Conference on Artificial Intelligence (AAAI)*, pages 29733–29735, 2025.
- [79] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. Gonzalez, C. Barrett, and Y. Sheng. SGLang: Efficient execution of structured language model programs. In Globerson et al. [21], pages 62557–62583.
- [80] C. Zhou, K. Liu, Y. Zhou, Q. Qiao, J. Gao, H. Zhang, I. Lu, N. Ho, L. Li, A. Lei, C. Cheng, S. Chiang, Y. Zeng, D. Zhang, R. Yang, K. Chen, A. Chen, P. Ma, W. Zhang, and C. Jin. LongStraw: Long-context RL beyond 2M tokens under a fixed GPU budget. Technical Report arXiv:2607.14952 [cs.LG], 2026.

A Appendix

A.1 Notation. Definitions

Here, we add some details missing in the main text.

A.1.1 Buffer Read/Write Access by `torch.scatter`, `torch.gather`

These linear operations are defined in https://docs.pytorch.org/docs/2.12/generated/torch.Tensor.scatter_.html. `scatter_` assigns values to entries in certain positions, `gather` extracts values of entries at certain positions. For reference, `arr.scatter_(dim, index, src)` requires `arr.ndim == index.ndim == src.ndim` and `index.shape == src.shape`, whereas `arr.shape` can differ on position `dim`. Say that `arr.ndim == 3`. Then:

$$\begin{aligned} \text{arr}[\text{index}[i, j, k], j, k] &= \text{src}[i, j, k] & | \dim == 0 \\ \text{arr}[i, \text{index}[i, j, k], k] &= \text{src}[i, j, k] & | \dim == 1 \\ \text{arr}[i, j, \text{index}[i, j, k]] &= \text{src}[i, j, k] & | \dim == 2 \end{aligned}$$

Also, if `res = arr.gather(dim, index)`, then:

$$\begin{aligned} \text{res}[i, j, k] &= \text{arr}[\text{index}[i, j, k], j, k] & | \dim == 0 \\ \text{res}[i, j, k] &= \text{arr}[i, \text{index}[i, j, k], k] & | \dim == 1 \\ \text{res}[i, j, k] &= \text{arr}[i, j, \text{index}[i, j, k]] & | \dim == 2 \end{aligned}$$

In our use case, we apply these operations to 4D arrays with `dim = 2`, so that `arr, src` are 4D, but `index` is 3D. This is done by broadcasting `index` along the final axis: `index.unsqueeze(-1).extend(-1, -1, -1, d_h)`.

A.2 Key-Value Cache Policies

In this section, we present additional details about KV cache policies used in our experiments.

A.2.1 Policy `smart_lastrec (slr)`

Recall the simple `lastrec (lr)` policy from Section 4, which keeps the last recent $N_C - \beta$ and first β tokens in the cache. One drawback of this policy is that β is fixed, while prompts often start with initial important control information of *variable* length. Our `smart_lastrec` policy is defined in terms of a regular expression for the end of this task control prefix, as well as some maximum prefix length $M_{\text{prefix}} < N_C$. When processing the first (prefill) chunk, we search for the first match (separately for each batch position b). If this results in a prefix length $M(b) \leq M_{\text{prefix}}$, this is used, otherwise $M(b) = M_{\text{prefix}}$. For subsequent chunks and token positions $t \geq N_C$, the policy is $\pi(b, h, t) = M(b) + \text{mod}(t - N_C, N_C - M(b))$.

We also implemented a generalization where a *range* $[M_0(b), M_1(b))$ is protected from eviction.¹¹ Here, $M_0(b)$ is chosen as the position of the first non-padding token, and $M_1(b)$ is

¹¹We allow for $M_1(b) < M_0(b)$, in which case the protected area is $[0, M_1(b)) \cup [M_0(b), N_C)$.

chosen as above. The idea is that initial padding tokens do not carry information and should not be attended to, so they can be evicted as soon as the cache is full. In this variant, if all tokens in the prefill chunk are padding for some b , the search for $[M_0(b), M_1(b))$ is shifted to subsequent chunks. The prefix case above is obtained with $M_0(b) = 0$. Surprisingly, in our experiments, the general variant did not improve over the prefix variant,¹² so that `smart.lastrec` in Section 4 is the prefix variant throughout.

A.3 Long-Context Benchmarks

A.3.1 Helmet

Our training and evaluation suite is derived from Helmet [69], a benchmark designed for inference-time evaluation of long-context language models. Helmet covers five capability categories across five context-length scales (8k to 128k tokens). Helmet provides only a small number of instances per task. For supervised fine-tuning, we adapted it as follows.

Instance Construction and Task Scope. We reconstruct each task from its upstream source data, following the original Helmet logic for forming contexts and controlling sequence length. We focus on the 64k and 128k context-length settings and include 10 tasks spanning five capability categories. Table 3 provides a summary of each task.

Split Separation. For each task we produce two non-overlapping partitions. The instances used in the original Helmet evaluation are reserved as a held-out *evaluation (test) set*. All remaining instances are collected into a *development set* used for training. For RAG tasks, we sample a single depth (difficulty) variant per query in the development set to prevent the model from memorising the same question paired with multiple distractor configurations. For InfiniteBench QA/MC, we remove in-context demonstrations from development instances because the demonstrations are drawn from the same small pool and would otherwise create data leakage during training.

Evaluation Metrics. Our evaluation metrics are taken from the code coming with Helmet (<https://github.com/princeton-nlp/HELMET/blob/main/utils.py>). Here, the output is a string generated by the model, the target is a string or a list of strings:

- SubEM: Depending on the dataset, the target can be a list of strings. We normalize the output (strip whitespace, quotes, and common phrases such as “Answer:”), map output and target(s) to lower-case. The value is 1 if at least one of the targets is a substring in the output, 0 otherwise.
- Accuracy: The target is a numerical value. We extract all numerical values from the output and find the value which occurs most often (with ties, the value is chosen which appears first). We then use exact match between this value and the target.
- ROUGE-F1: The target is a string. We compute ROUGE-N precision/recall/F1 between output (after normalization as in SubEM) and target.

Category	ID	Source	Metric	Dev	Eval
RAG	nq	Natural Questions	SubEM	893	600
	trivia_qa	TriviaQA	SubEM	876	600
	pop_qa	PopQA	SubEM	192	600
	hotpot_qa	HotpotQA	SubEM	787	300
Many-shot ICL	trec_coarse	TREC	Accuracy	1000	500
	nlu	SNIPS NLU	Accuracy	2094	500
	clinc150	CLINC150	Accuracy	2600	500
Long-doc QA	inf_qa	InfiniteBench QA	ROUGE-F1	251	100
	inf_mc	InfiniteBench MC	Accuracy	129	100
Synthetic Recall	json_kv	JSON-KV	SubEM	500	100

Table 3: Overview of the 10 Helmet tasks. *Dev* and *Eval* denote the number of instances in the training and evaluation partitions, respectively, at a single context-length setting.

In what follows, we provide details for the different tasks.

Retrieval-augmented Generation (RAG). Each instance consists of a natural-language question, one or more gold passages, and a pool of hard-negative distractors. The context is formed by inserting gold passages at a random depth among the distractors, and the whole context is truncated to the target length. *Natural Questions* (NQ) uses real Google search queries paired with Wikipedia passages. *TriviaQA* uses trivia questions authored with independently collected evidence. *PopQA* focuses on long-tail, entity-centric questions generated from Wikidata triples; we filter the evaluation set to queries whose subject entities fall below a popularity threshold of 3. *HotpotQA* requires multi-hop reasoning across two gold passages. Each of the six depth variants of a query places the gold passage(s) at a different relative position in the distractor pool. All four tasks are evaluated with substring exact match (SubEM).

Many-shot In-context Learning. Three intent/question-type classification datasets test the model’s ability to exploit many labelled demonstrations placed entirely within the context window. Unlike most other tasks, where demonstrations serve only as formatting guides, here they carry essential semantic information: each demonstration encodes a (text, ordinal-label) pair, and the label-to-class mapping is only recoverable by reading the demonstrations. *TREC Coarse* has 6 question-type classes; *SNIPS NLU* has 68 intent classes; *CLINC150* has 151 intent classes. The number of demonstrations is calibrated to fill the context window while maintaining an approximately balanced class distribution across shots.

Long-document QA. Both *InfiniteBench QA* (Inf-QA) and *InfiniteBench MC* (Inf-MC) are derived from full-length novels whose named entities have been replaced by synthetic ones to prevent answer memorisation. The source document typically exceeds the target context window, so it is truncated to fit. Inf-QA is open-ended, evaluated by ROUGE-F1; Inf-MC is a 4-way multiple-choice variant evaluated by accuracy.

¹²This needs further analysis. Maybe some initial padding tokens are used as so-called ”attention sinks” [65].

Synthetic Recall. *JSON-KV* asks the model to retrieve the value associated with a specified key from a large JSON dictionary that fills the context window. This task is evaluated with SubEM and serves as controlled probes of the model’s ability to locate and copy specific information over very long spans.

A.4 Gradient Computation

In this section, we provide additional details about our long-context gradient computation method from Section 3.2.

A.4.1 Chunks, Cells, CPU Offloading

Recall that for long-context inference or fine-tuning with cache length N_C , we split a sequence of length $N > N_C$ into $1 + \lceil (N - N_C)/S \rceil$ *chunks*, the first (prefill) chunk of length N_C , subsequent chunks of length $S < N_C$. The chunk size S is chosen according to a latency-vs-accuracy trade-off, it is in general much shorter than N_C (Section 3.3). We also group chunks into *cells*. The first cell consists of the prefill chunk alone, subsequent cells group $k = \lfloor \alpha N_C / S \rfloor$ of S -length chunks, where $\alpha > 0$ is a hyperparameter which defaults to $\alpha = 1$.

In our gradient computation method, *PyTorch autograd* is run on cells. While S is chosen also with accuracy in mind, the choice of k and α is determined by efficiency (both runtime and memory) only. The idea is that the *autograd* GPU memory requirements are on the order of one KV cache buffer, if we exploit the linear recurrence (see Section 3.2 and below). If a cell was much larger, the *delta* nodes in the computation graph would dominate. The α parameter is adjusted so to not run out of memory. Empirically, a smaller chunk size S necessitates a smaller α , likely due to overhead in *autograd* (for constant α , smaller S means larger k , so a larger computation graph). In our experiments, we chose $\alpha = 1$ for $S = 2048$, $\alpha = 0.75$ for $S = 1024$, and $\alpha = 0.1$ for $S = 128$, for a cache length of $N_C = 32768$ and 40 GB of GPU memory.

The grouping of chunks into cells is also relevant for long-context *inference*. Recall that L denotes the number of layers of our model, and layer inputs are of shape (B, N, d) , where $d = H_q \cdot d_h$ is the model embedding dimension. For large N , not even the inputs to one layer can be kept in GPU memory. This has implications for how inference is computed and what can be offloaded to CPU memory at which point during the process (our library supports CPU offloading of KV cache buffers during the forward passes, as well as CPU offloading of weights during the backward pass, but this is not used in the experiments reported here).

Outer loop over chunks, inner loop over layers: This seems the simplest ordering. Also, layer inputs can be kept in GPU memory, with outputs overwriting inputs. A major drawback is that either all KV cache buffers need to be kept in GPU memory, or they need to be read from and written back to CPU memory frequently. When KV cache buffers are stored in quantized form, the quantization computation can be considerable. For long-context inference, this ordering is not suitable.

Outer loop over layers, inner loop over chunks: With this ordering, KV cache buffers (and even model weights) can be offloaded to CPU for all but the currently active layer. A drawback of this ordering is that layer inputs and outputs cannot be kept in GPU memory

in total, so need to be offloaded eventually. Still, this ordering is much better than the previous one.

Outer loop over cells, middle loop over layers, inner loop over chunks per cell:

This ordering provides a good compromise between the two previous ones. Layer inputs and outputs can be kept in GPU memory, while KV cache buffers can be quantized and/or offloaded to CPU, which happens much less frequently. We use this ordering in our implementation, also because CPU offloading of KV cache buffers is needed during gradient computation, so we can just reuse this code.

A.4.2 Summary of Method

Here, we present a detailed summary of our gradient computation technique. Recall that we process a batch of B sequences of token length N , and that caches in each layer have length N_C . For $N \leq N_C$, our method reduces to standard training, so assume that $N > N_C$. For simplicity, we assume that caches in all layers have the same length N_C . This is easy to relax, and our implementation does so.

At the top level, our method runs a forward pass followed by a backward pass, just like standard code. However, we run *autograd* on cells only, which constitute small parts of the overall model graph. As with activation checkpointing [25], this means we need to run forward passes over the model three times (instead of just once). The first two passes are run in non-autograd mode, checkpointing (so called) boundary information to CPU memory. The third pass is part of the autograd runs on cells, which consume boundary information as inputs.

We need some notation. Let $\mathcal{L}$ denote the training loss for the current batch. Layers are indexed by $l = 0, \dots, L - 1$, cells by $c = 0, \dots, N_{\text{cells}} - 1$. $\mathbf{X}_l$ are inputs to layer l , of shape (B, N, d) , and $\mathbf{X}_L$ is the top layer output. Moreover, $\mathbf{X}_{l,c}$ denotes the slice of $\mathbf{X}_l$ along axis 1 corresponding to the cell. Finally, let $\mathbf{K}_{l,c}$ denote the KV cache buffers¹³ of shape $(2, B, H_k, N_C, d_h)$ at the input of cell $c > 0$ in layer l .

Forward pass 1. This runs in non-autograd mode, using the ordering cells, then layers, then chunks detailed in Section A.4.1. Alongside:

- Store KV cache replay log in each layer, containing the decisions $\{\pi(b, h, t)\}$.
- Checkpoint layer inputs $\mathbf{X}_l$ to CPU memory for each layer $l = 0, \dots, L - 1$. Our implementation allows to quantize them in order to save CPU memory and CPU-GPU transfer time, but this is not activated in our experiments, since the time is subdominant. We also checkpoint top layer outputs $\mathbf{X}_L$.

Backward pass. This runs backwards over layers. In each step, we compute gradients for weights in layer l using activation checkpointing. We start with computing head gradients $\partial\mathcal{L}/\partial\mathbf{X}_L$ based on top layer outputs $\mathbf{X}_L$, writing them to CPU (in fact, head gradients $\partial\mathcal{L}/\partial\mathbf{X}_l$ overwrite $\mathbf{X}_l$ on CPU). Next, we iterate over layers $l = L - 1, \dots, 0$:

- Forward pass 2 for layer l (non-autograd mode). Runs over cells $c = 1, \dots, N_{\text{cells}} - 1$, computing the cache buffers $\mathbf{K}_{l,c}$ and storing them to CPU. These are quantized in

¹³These are really two tensors (keys, values), but we concatenate them into one for notational simplicity.

order to save CPU memory and CPU-GPU transfer time, and we overwrite the cache buffer checkpoints from previous layer $l+1$. Cache decisions are replayed from the log. Note that we could checkpoint *all* $\mathbf{K}_{l,c}$ during forward pass 1, but this would require L times more CPU memory, and the extra time for forward pass 2 is subdominant. Also note that we load inputs $\mathbf{X}_{l,c}$ to forward pass 2 to GPU cell by cell: the whole $\mathbf{X}_l$ does not fit in GPU memory (see Section A.4.1).

- Run *autograd* on each cell, iterating from right to left, $c = N_{\text{cells}} - 1, \dots, 0$. For each cell c , we load layer inputs $\mathbf{X}_{l,c}$ (bottom), layer head gradients $\partial\mathcal{L}/\partial\mathbf{X}_{l+1,c}$ (top) and incoming cache buffers $\mathbf{K}_{l,c}$ (left; only for $c > 0$) from CPU, while cache buffer head gradients $\partial\mathcal{L}/\partial\mathbf{K}_{l,c+1}$ (right; only for $c < N_{\text{cells}} - 1$) are kept in GPU memory. Cache decisions are replayed from the log. *autograd* works as follows:
 - Forward pass 3 for cell (l, c) , in autograd mode. Whenever a KV cache buffer node (**keys**, **values**) is created, we store an *annotation* in a list. In the `pack_hook` function, we match arguments of the right shape against all annotation. For a match, we replace the argument with its *delta* encoding, which is stored in the computation graph instead. See Section A.4.3 for details.
 - Backward: When *PyTorch* traverses the computation graph in reverse order, it calls `unpack_hook` for each node. For each *delta* encoding, we play the recurrence (4) backwards, replacing **keys'** by **keys** or **values'** by **values**.

The outcome of *autograd* on cell (l, c) are gradients w.r.t. layer weights (which are accumulated), a head gradient $\partial\mathcal{L}/\partial\mathbf{X}_{l,c}$ (written to CPU, overwriting $\mathbf{X}_{l,c}$), and a head gradient $\partial\mathcal{L}/\partial\mathbf{K}_{l,c}$ which replaces the previous one (for $c > 0$). At the end of the loop over cells, gradients w.r.t. layer weights are complete, and the head gradient $\partial\mathcal{L}/\partial\mathbf{K}_l$ is on CPU, so the layer below can be addressed (or, for $l = 0$, gradients w.r.t. input embeddings can be computed based on $\partial\mathcal{L}/\partial\mathbf{K}_0$).

A.4.3 Exploiting Linear Recurrence of KV Cache Buffers

Recall the recurrence between KV cache buffers for subsequent chunks from Section 3.2. We can exploit this recurrence by asking *PyTorch* to store `delta_key` instead of **keys** and `delta_value` instead of **values** in the computation graph, restoring the latter from the former during backward using (4).

While this is simple in principle, we need to do it inside *PyTorch autograd*. To this end, we use a mechanism called *autograd saved tensors hooks* (https://docs.pytorch.org/tutorials/intermediate/autograd_saved_tensors_hooks_tutorial.html). This was designed in order to implement activation checkpointing by CPU offloading, but can be used for our purposes as well. It works by allowing the specification of two functions:

- `pack_hook(x) → p(x)`: When building its computation graph during forward, this function is called for every array $\mathbf{x}$ PyTorch plans to store in the computation graph. It then stores $\mathbf{p(x)}$ in the graph instead of $\mathbf{x}$.
- `unpack_hook(p) → x(p)`: When traversing the computation graph in reverse order during backward, PyTorch calls this function for every array $\mathbf{p}$ stored in the graph. It then uses $\mathbf{x(p)}$ instead of $\mathbf{p}$.

A major difficulty for us is the non-selectiveness of this mechanism. We do not want to pack all arrays stored in the graph, but only specific ones: the KV cache buffers (for all other nodes, we just pass through $p(x) = x$ and $x(p) = p$). But `pack_hook(x)` just takes a `torch.Tensor` argument, there is no obvious way for tagging the nodes we want. Also, there is some delay between a node being created in the forward pass and `pack_hook(x)` being called for it, we even detected some differences in the relative ordering. Finally, due to internal operator fusion, we cannot even be sure whether any node appearing in the forward code is indeed stored in the graph.

Our implementation maintains an *annotation list*, which is appended to during the forward code, while entries are removed during `pack_hook` calls. Whenever a KV cache buffer update in the form of a statement `keys' = scatter(keys, index, key_new)` is passed in the forward code, we compute `delta_key = gather(keys, index)`, appending an *annotation* containing `(index, delta_key)` and some meta-data to the list. Here, `delta_key` serves a double role. First, it is needed to reconstruct `keys` from `keys'` in `unpack_hook`. Second, it serves¹⁴ as a "fingerprint" of `keys`. Namely, when `pack_hook(x)` is called, we need to match the argument `x` against annotations. This is first done by shape, filtering out most calls. Next, for any annotation `(index, delta_key)`, we check whether `gather(x, index) = delta_key`. If so, we return $p(x)$ containing `delta_key` and remove the annotation from the list. If there is no match, we return $p(x) = x$.

For `unpack_hook(p)`, we reconstruct the sequences `keys` and `values` in reverse order. If `p` is not a packed object, we return $x(p) = p$. Otherwise, we check the chunk number stored with `p` against the current state `(keys', values')`. If this fits, we reconstruct `keys` from `(keys', delta_key)` or `values` from `(values', delta.value)`, using (4). The new buffer overwrites the old one.

Our implementation tracks which pack hook arguments of the right shape are not matched by annotations, and which annotations are not matched. Both events do happen, but at a low rate. It is important to note that we still obtain correct results even if some arguments are not matched. This just means that a bit more GPU memory is being used. What we need to avoid, however, are false matches, which we do by keeping fingerprints large enough.

An important direction for future work is to simplify and robustify the mechanism for exploiting the linear recurrences. We tried several simplifications. One assumes that KV buffer nodes are created in exactly the same order as `pack_hook(x)` calls. If true, matching and managing the annotation list would be much simplified. Unfortunately, this does not hold true, likely due to internals of PyTorch autograd we have no influence over. In fact, the instruction `keys' = scatter(keys, index, key_new)` need not even trigger a call of `pack_hook` with $x = keys'$. The array `keys'` is processed further inside the SDPA code, and PyTorch may use some form of operator fusion. The simplest solution would be to tag each KV buffer node during creation in a way that allows us to recognize the tag in a pack hook argument `x`. However, we did not find a way to do that yet.

¹⁴If `index` is small, we extend the fingerprint by additional random positions in order to avoid false matches.

A.5 Experimental Results

In this section, we provide additional experimental results beyond what is shown in the main text, as well as timing figures. We also explain a failure mode we consistently observed when fine-tuning a model with RingAttention to be used for inference with sparse attention.

A.5.1 Additional Results

	64k datasets				128k datasets			
	nq	tri_qa	hot_qa	pop_qa	nq	tri_qa	hot_qa	pop_qa
exact	-	-	-	-	-	-	-	-
lr _{2k}	47.5	80.2	57.7	53.8	35.3	70.5	37.0	40.2
slr _{2k}	45.3	80.2	55.3	52.0	37.5	69.7	34.7	34.8
h2o _{2k}	42.8	70.8	44.7	59.0	20.5	63.2	12.3	27.3
h2o _{2k} ^{no}	46.0	82.2	52.7	59.3	42.2	78.5	34.3	39.3
h2o _{2k} ^{or}	44.2	74.7	47.7	61.0	43.7	79.8	25.0	43.3
lr _{1k}	47.5	80.2	59.7	54.0	35.8	70.2	37.7	37.5
slr _{1k}	49.0	79.0	55.0	54.3	37.3	67.3	32.0	35.7
h2o _{1k}	43.2	71.7	41.7	59.8	21.0	62.8	13.0	27.0
h2o _{1k} ^{no}	47.0	81.2	50.0	56.7	40.0	79.8	32.7	42.3
h2o _{1k} ^{or}	47.5	76.3	46.7	58.2	41.8	79.8	23.7	39.3

Table 4: Results for long-context inference with 5 KV cache policies and chunk sizes $2048 = 2k, 1024 = 1k$ (rows). Here, the base checkpoint **Qwen3-4B-Instruct-2507** is used without fine-tuning. The first row *exact* is for exact inference (sequence parallelism). We show `sub_exact_match` values on test splits for different Helmet datasets **nq**, **trivia_qa**, **hotpot_qa**, **pop_qa**, limiting sequence lengths to 64k or 128k tokens.

In Table 4, we provide results on Helmet 64k and 128k datasets for base checkpoints **Qwen3-4B-Instruct-2507** (no fine-tuning). They should be related to results in Table 1, where the base model was trained by our method (columns *us*) or by sequence parallelism (columns *sp*).

In Table 5, we provide results on Helmet 64k and 128k datasets for setups not covered in the main text (see Table 1). `slr128`, `h2o128`, `h2o128no`, `h2o128or` use chunk size $S = 128$. This runs significantly longer than $S \in \{1024, 2048\}$ used in the main text experiments, but allows the KV cache policy to make decisions 8 or 16 times more frequently. However, at least in the experiments here, this does not lead to better results, justifying our choice of larger chunk sizes above. We also ran experiments with Q-Hitter [77], where KV cache buffers are quantized (to 8 bits in our experiments) and the decision score is a convex combination of (3) and a term quantifying the quantization error. The results are consistently worse than for the H2O variants.

	64k datasets			
	nq	tri_qa	hot_qa	pop_qa
slr ₁₂₈	38.8	66.2	51.7	41.0
h2o ₁₂₈	44.5	65.3	55.7	54.3
h2o ₁₂₈ ^{no}	49.0	74.3	55.0	56.3
h2o ₁₂₈ ^{or}	46.8	67.3	48.3	52.3
qh2o _{2k}	37.5	64.0	40.3	53.7
qh2o _{2k} ^{no}	40.2	65.5	46.3	53.5

Table 5: Results for long-context inference with setups not covered in the main text. We show `sub_exact_match` values on test splits for different Helmet datasets `nq`, `trivia_qa`, `hotpot_qa`, `pop_qa`, limiting sequence lengths to 64k or 128k tokens. `slr128`, `h2o128`, `h2o128no`, `h2o128or` use chunk size $S = 128$. `qh2o2k` and `qh2o2kno` are variants of Q-Hitter [77].

	nq	tri_qa	hot_qa	pop_qa
exact	258.38 (15.14)	266.05 (11.31)	262.53 (8.18)	236.74 (21.44)
lr _{2k}	326.07 (21.86)	333.28 (16.56)	330.76 (22.49)	312.29 (27.71)
slr _{2k}	323.84 (21.19)	333.03 (16.19)	330.29 (22.55)	310.73 (24.46)
h2o _{2k}	330.83 (21.98)	344.83 (16.70)	336.79 (23.13)	316.02 (27.66)
h2o _{2k} ^{no}	331.48 (22.10)	341.71 (16.65)	337.77 (23.12)	317.60 (25.16)
h2o _{2k} ^{or}	331.83 (22.25)	344.85 (16.89)	338.46 (23.25)	316.94 (25.34)
lr _{1k}	364.10 (23.28)	374.80 (18.62)	375.38 (26.38)	343.96 (27.99)
slr _{1k}	362.84 (23.20)	370.55 (18.66)	371.18 (25.77)	344.77 (27.65)
h2o _{1k}	378.85 (24.88)	385.19 (20.21)	382.83 (26.94)	359.51 (32.00)
h2o _{1k} ^{no}	378.61 (24.99)	385.29 (19.32)	388.77 (27.69)	357.95 (29.02)
h2o _{1k} ^{or}	378.65 (24.79)	385.64 (19.46)	382.52 (26.91)	358.99 (31.74)

Table 6: Running time figures for training update step, for Helmet 128k datasets (columns), 5 KV cache policies and chunk sizes $2048 = 2k$, $1024 = 1k$ (rows). Batch size 8, running on 4 devices. The step from 2k to 1k is 11% to 13% more expensive for `lr`, `slr`, 12% to 14% more expensive for `h2o` variants. The step from `lr`, `slr` to `h2o` variants is 2% to 3% more expensive for 2k, 3% to 4% more expensive for 1k.

A.5.2 Running Time Figures

In this section, we present running time figures. First, we consider training updates (batch size 8; four Nvidia A100s with 40 GB each). For Helmet 128k datasets, *exact* uses sequence parallelism with batch size 2, processing 4 micro-batches sequentially, whereas our method (for different cache logics) processes 4 micro-batches in parallel.

First, our method is about 30% more expensive than *exact* for chunk size $S = 2048$ (2k). Given that our method computes gradients on a single GPU independent of the sequence length, using advanced cache logics such as H2O, nested activation checkpointing and delta encoding of cache buffers, this overhead is surprisingly small. Reasons for the overhead are explained in Section 3.3. The gap can likely be narrowed further by operator fusion,

increasing the chunk size *autograd* is operating with. Next, we would expect $S = 1024$ (1k) to run slower than $S = 2048$ (2k), because more chunks need to be processed sequentially; and H2O policies to run slower than `lr`, `slr`, because summed attention weights are required, and scores need to be computed and sorted. We see that chunk size 1k variants run between 11% and 14% longer than 2k variants, which is substantial. On the other hand, H2O variants are only between 2% to 4% slower than `lr`, `slr`. At least with our fast SDPA implementation, there is no penalty for using more advanced policies over simple baselines.

A.5.3 Analysis of Errors

In this paper, we compare different ways of fine-tuning a model to be used with sparse attention inference under different KV cache logics: training by sequence parallelism (*sp*) versus training with the new, resource-efficient technique developed here (*us*). While for datasets `nq`, `trivia_qa`, `hotpot_qa`, `pop_qa` coming with the *SubEM* metric, results are inconclusive (see Table 1), *us* strongly outperforms *sp* on datasets `trec_coarse`, `nlu`, `clinc150`, `inf_qa`, `inf_mc`, `json_kv`, where the metric is mostly *Accuracy* (see Table 2). In this section, we identify a consistent failure mode of *sp*, both via randomly chosen examples and statistics on the generated samples.

Recall details about Helmet datasets from Section A.3.1. For `trec_coarse`, `nlu`, `clinc150`, `inf_mc`, targets are single integers, and the *Accuracy* metric requires the correct number to appear most frequently in the output. For `nq`, `trivia_qa`, `hotpot_qa`, `pop_qa`, `json_kv`, targets are lists of noun phrases of one or few words, and the *SubEM* metric requires one of the targets to be a substring of the output. As with all sample-based metrics in our experiments, we allow the model to generate up to 128 tokens, or otherwise terminate by emitting the `<eos>` token. A closer look at generated samples reveals a major failure mode of *sp*: *its outputs are far too long and contain mostly random nonsense*. Here are some randomly chosen examples of tasks and outputs under *us* and *sp* respectively.

`trec_coarse` / `smart_lasttrec_1024`:

```

- idx: 10
  raw_target: "2"
  exact_match_us: 1
  output_us: "2"
  exact_match_sp: 0
  output_sp: " 2
    What is the average life expectancy of a female in Ireland in 1996 ?
    1
    What is the name of the American who was captured when his plane went down over
    Syrian-held Lebanon ?
    1
    0
    What is the full name of the PLO ?
    1
    3
    What is the difference between a college and a university ?
    1
    4
    What is the origin of the term ‘ ‘ The Brooklyn Side ‘ ’ ’ ’ in bowling ?
    1
    4
    What is the origin of the word ‘ ‘ assassinate ‘ ’ ’ ’ ?
    1
    4
    What does the abbreviation cwt. ? mean ?
    1
    3
    What does the"
- idx: 318
  raw_target: "5"
  exact_match_us: 1
  output_us: "5"
  exact_match_sp: 0
  output_sp: " 1
    What does the acronym CPR mean ?

```


json_kv / h2onorm_1024:

```
- idx: 81
  raw_target: d823df41-5af0-4f11-925d-d0e8018cc5be
  sub_exact_match_us: 1
  output_us: d823df41-5af0-4f11-925d-d0e8018cc5be
  sub_exact_match_sp: 0
  output_sp: ' 923928d-2e95-426c-8893-b5e80880a88c
    json
    {"c": "11984809639058896", "a": "923928d-2e95-426c-8893-b5e80880a88c", "b": "0e2b980e-da28-4af1-8896-3f0c '
- idx: 51
  raw_target: d8cdc4a6-e37e-4cd3-b603-ad33233518e9
  sub_exact_match_us: 0
  output_us: d8cdc4a6-e37e-4cd3-b6d3-ad33233b18e9
  sub_exact_match_sp: 0
  output_sp: " 033a8782-23d2-488c-ae99-488f80bbc7d6\n\nKey: 9c71e7e3-9f60-47f4-baaa-42dbca3e2715:\n
    \ \ "d477445a-0f1b-4729-b586-10cbca0a5ba3\"," \n      \ "9f71e7e3-9f60-47f4-baaa-4"
```

nq:

```
- idx: 418
  raw_target:
    - six
    - eight
  sub_exact_match_us: 1
  output_us: "six"
  sub_exact_match_sp: 0
  output_sp: " 4 hoops are used in a game of croquet . ( 2 blue , 1 red"
- idx: 419
  raw_target:
    - six
    - eight
  sub_exact_match_us: 0
  output_us: "four"
  sub_exact_match_sp: 0
  output_sp: " 20 hoops ( 10 per side ) are used in a game of croquet ."
```

pop_qa:

```
- idx: 275
  raw_target:
    - Paraguay
    - Republic of Paraguay
    - py
    - "\U0001F1F5\U0001F1FE"
    - Heart of South America
  sub_exact_match_us: 0
  output_us: "Peru"
  sub_exact_match_sp: 0
  output_sp: " Peru
    Question: What is the capital of Peru?
    Answer: Lima
    Question: In what region"
- idx: 273
  raw_target:
    - Paraguay
    - Republic of Paraguay
    - py
    - "\U0001F1F5\U0001F1FE"
    - Heart of South America
  sub_exact_match_us: 0
  output_us: "Peru"
  sub_exact_match_sp: 0
  output_sp: " Peru
    Question: What is the name of the Peruvian city where the National Library is
    located?"
```

Most other examples we inspected reveal the same failure mode. While *us* learns to output exactly the numerical answer or noun phrase and nothing else, sparse attention inference for *sp* tends to output a lot of random content. Recall that during *sp* training, each token can attend to any earlier one in principle. Plugging in a cache eviction logic afterwards seems to *diminish the model’s ability to correctly stop generation*.¹⁵ Sometimes, the first number in the output is correct, but is followed by many others in the output. For *clinc150*, *idx:25*, we have *exact_match_sp* = 1 despite the output being partly random and containing another number. For *nlu*, *idx:245*, the correct answer 27 appears most frequently in

¹⁵For most cache logics, inference with *sp* checkpoints does stop with `<eos>` for some samples, emitting sometimes as few as 5 or 7 tokens. Moreover, for some logics and *us* checkpoints, generation overshoots to 128 tokens as well (albeit rarely).

nonsense output. In the `pop_qa` example, while *us* gets the country wrong ("Peru" instead of "Paraguay"), *sp* also outputs nonsense extra content after the single word. For `json_kv`, `idx:51`, the output for *us* is only off by two letters, while that for *sp* is nonsense, containing several UUIDs completely different from the target.

	trn	slr _{1k}		h2o _{1k} ^{no}		h2o _{1k} ^{or}	
		<i>R</i>	<i>p</i> ₁₂₈	<i>R</i>	<i>p</i> ₁₂₈	<i>R</i>	<i>p</i> ₁₂₈
nq	us	1.1±0.8	0.0±0.0	1.1±1.0	0.0±0.0	1.1±2.7	0.2±4.1
	sp	35.5±24.1	99.5±7.1	35.5±21.6	100.0±0.0	36.2±22.1	99.3±8.1
	no	35.2±22.0	97.7±15.1	35.8±22.8	99.2±9.1	34.9±22.0	97.8±14.6
trivia_qa	us	1.2±0.8	0.0±0.0	1.3±1.0	0.0±0.0	1.1±0.7	0.0±0.0
	sp	35.4±24.1	97.3±16.1	38.2±25.2	99.2±9.1	42.0±24.4	96.7±18.0
	no	43.4±28.5	87.8±32.7	46.5±28.9	96.0±19.6	48.2±30.4	95.7±20.4
hotpot_qa	us	1.0±0.5	0.0±0.0	1.0±0.6	0.0±0.0	1.3±1.4	1.0±9.9
	sp	39.2±31.8	93.0±25.5	40.9±31.4	98.3±12.8	41.4±31.1	99.0±9.9
	no	41.0±30.9	91.7±27.6	41.3±31.1	98.0±14.0	41.4±31.1	98.7±11.5
pop_qa	us	1.1±0.6	0.0±0.0	1.1±0.5	0.0±0.0	1.0±0.4	0.0±0.0
	sp	53.8±24.9	98.8±10.7	54.4±24.7	99.5±7.1	55.0±24.7	98.7±11.5
	no	58.9±29.3	93.5±24.7	60.0±30.9	98.8±10.7	58.2±28.9	96.8±17.5
trec_coarse	us	1.0±0.0	0.0±0.0	1.0±0.0	0.0±0.0	1.0±0.0	0.0±0.0
	sp	127.6±6.1	99.6±6.3	127.3±8.4	99.0±9.9	128.0±0.1	99.6±6.3
	no	128.0±0.0	100.0±0.0	128.0±0.0	100.0±0.0	128.0±0.1	99.4±7.7
nlu	us	1.0±0.1	0.0±0.0	1.0±0.1	0.0±0.0	1.0±0.2	0.0±0.0
	sp	70.4±21.1	70.6±45.6	69.0±21.7	45.6±49.8	14.6±18.8	8.4±27.7
	no	71.7±20.8	100.0±0.0	71.7±20.8	100.0±0.0	71.7±20.8	99.8±4.5
clinc150	us	1.0±0.1	0.0±0.0	1.0±0.1	0.0±0.0	1.0±0.1	0.0±0.0
	sp	30.2±28.6	43.8±49.6	32.6±31.0	49.8±50.0	57.6±32.2	87.6±33.0
	no	59.9±19.7	100.0±0.0	59.9±19.7	100.0±0.0	60.1±20.5	100.0±0.0
inf_qa	us	1.1±0.8	0.0±0.0	1.2±0.8	0.0±0.0	1.2±0.8	0.0±0.0
	sp	45.7±32.4	100.0±0.0	45.7±32.4	99.0±9.9	45.8±32.3	97.0±17.1
	no	45.5±32.3	95.0±21.8	45.7±32.4	99.0±9.9	45.7±32.4	96.0±19.6
inf_mc	us	1.0±0.0	0.0±0.0	2.3±12.6	1.0±9.9	1.0±0.0	0.0±0.0
	sp	128.0±0.0	100.0±0.0	128.0±0.0	100.0±0.0	127.9±1.2	98.0±14.0
	no	128.0±0.0	100.0±0.0	128.0±0.0	100.0±0.0	128.3±3.6	97.0±17.1
json_kv	us	1.1±0.1	0.0±0.0	1.0±0.0	0.0±0.0	3.7±1.0	85.0±35.7
	sp	4.1±0.3	100.0±0.0	4.1±0.3	100.0±0.0	4.1±0.3	98.0±14.0
	no	4.1±0.3	100.0±0.0	4.1±0.3	100.0±0.0	4.1±0.3	100.0±0.0

Table 7: Token length statistics of generated samples for 10 Helmet datasets (of context width 128k) and 3 cache logics. *trn* denotes model checkpoint being used: *us* uses our novel method with the same cache policy in place, *sp* is using sequence parallelism, *no* is the base checkpoint `Qwen3-4B-Instruct-2507` (no fine-tuning). *R* is based on the ratio of output length to target length (in tokens), *p*₁₂₈ (in percent) is the fraction of outputs of maximal size 128 (means, and stddevs over all test set samples).

In order to quantify the prevalence of this failure mode across all datasets and setups, we

nq		tri_qa		hot_qa		pop_qa		trec_c	
R	p_{128}	R	p_{128}	R	p_{128}	R	p_{128}	R	p_{128}
1.1±1.1	0.0±0.0	1.0±0.6	0.0±0.0	1.1±0.6	0.0±0.0	1.0±0.4	0.0±0.0	1.0±0.0	0.0±0.0
nlu		clc150		inf_qa		inf_mc		json_kv	
R	p_{128}	R	p_{128}	R	p_{128}	R	p_{128}	R	p_{128}
1.0±0.1	0.0±0.0	1.0±0.0	0.0±0.0	1.2±0.9	0.0±0.0	1.0±0.0	0.0±0.0	1.0±0.0	0.0±0.0

Table 8: Token length statistics of generated samples for 10 Helmet datasets (of context width 128k) for training and inference with exact attention (sequence parallelism).

use two statistics, estimated over all samples generated for each dataset and cache logic:

$$R = \frac{\text{len}(\text{output})}{\text{len}(\text{target})}, \quad p_{128} = \mathbb{I}_{\{\text{len}(\text{output})=128\}}.$$

Here, $\text{len}(\cdot)$ denotes length in tokens, and samples are capped at 128 tokens. For some datasets, the targets are a list, in which case **target** is the longest entry appearing as substring in **output**, or the longest entry otherwise. Statistics for 10 Helmet datasets and 3 setups are shown in Table 7. With *us*, we have $R \approx 1$ across datasets and setups: outputs are close in length to targets, the model learned the desired output type and stops generation properly. But with *sp*, R tends to be large, and p_{128} is often close to 100%. This is not a property of the *sp* checkpoints. As seen in Table 8, $R \approx 1$ and $p_{128} \approx 0$ if exact inference is used. Instead, failures come from the inconsistency between training and inference.

If we relate numbers in Table 7 with good results in Table 1 for *sp* and in Table 4 for *no*, this points to a shortcoming of the *SubEM* metric used for these datasets. Insensitive to any type and amount of nonsense extra output, *it only requires the target to be contained in the output*, without requiring a definite way to extract the substring. Once such a requirement is added, as in *Accuracy*, performance for *sp* and *no* plummets. In any case, by not even providing succinct outputs (a very clear signal in the data), *sp* clearly does not behave satisfactory. While the tolerance of *SubEM* (and also *Accuracy*, to a lesser extent) to any amount of extra output is intended to not disadvantage LLMs (which "sugar-coat" answers in longer sentences), it makes the metrics blind to extra nonsense content returned. We should at least ask for a deterministic way to extract the response from the output.

A.6 Computing Summed Attention Weights in SDPA

As noted in Section 3.1.1 and Section 3.3.1, KV cache policies like H2O or related ones need summed attention weights $\sum_i m_{b,h,i,j}$ for each (b, h, j) . This array of shape (B, H_q, N_k) can be obtained as byproduct of SDPA, which returns $\mathbf{Y}$ of shape (B, H_q, N_q, d_h) . Note that summed attention weights are smaller than attention outputs, so there is a priori no reason for not returning them. However, all fast SDPA codes we know of, do not return this information. FlexAttention [13] can return log-sum-exp values $\log(\mathbf{A}\mathbf{1}_{N_k})$, where $\mathbf{A} = \text{mask}(d_h^{-1/2}\mathbf{Q}\mathbf{K}^T)$ is the argument of `softmax`, likely because this is directly computed during FlashAttention [9].

Our implementation contains *Triton* code for computing summed attention weights alongside a FlashInfer SDPA kernel [68]. This is an add-on, and it would be better if leading

SDPA codes returned summed attention weights directly. In this section, we detail how this can be done (even though we have not implemented this). We also show how to compute them with FlexAttention [13], using two calls instead of one. This is contained in our implementation as baseline.

A.6.1 FlashAttention for Summed Attention Weights

FlashAttention works by essentially computing the attention weights tensor $\mathbf{M}$ in blocks, using a lattice tiling along the query and the key axes. It can be understood as *map-reduce*, where *map* is independent per cell. More precisely, the full attention weights have shape (B, H_q, N_q, N_k) . In the following, we drop (B, H_q) , treating them as "batch" dimensions. Use cell indices (r, s) and index ranges $I(r)$, $J(s)$, so that the union of all $I(r)$ covers $\{0, \dots, N_q - 1\}$ and the union of all $J(s)$ covers $\{0, \dots, N_k - 1\}$. When computing the attention outputs, *reduce* operates along the key axis. Define an additional auxiliary tensor of shape (B, H_q, N_q) , with values

$$\lambda_{r,s} := \log \left(\exp \left(\mathbf{A}_{I(r), J(s)} \mathbf{1}_{|J(s)|} \right) \right), \quad \mathbf{A}_{I,J} := \text{mask} \left(d_h^{-1/2} \mathbf{Q}_I, \mathbf{K}_{J,}^T \right).$$

Reduction works as:

$$\begin{aligned} \lambda_{r,s_1 \oplus s_2} &= \max \{ \lambda_{r,s_1}, \lambda_{r,s_2} \} + \log 1p \left(\exp \left(-|\lambda_{r,s_1} - \lambda_{r,s_2}| \right) \right), \\ \mathbf{Y}_{r,s_1 \otimes s_2} &= (\text{diag} \exp (\lambda_{r,s_1} - \lambda_{r,s_1 \otimes s_2})) \mathbf{Y}_{r,s_1} + (\text{diag} \exp (\lambda_{r,s_2} - \lambda_{r,s_1 \otimes s_2})) \mathbf{Y}_{r,s_2}. \end{aligned}$$

We can now run *map* independently for all (r, s) , then *reduce* along s for all r .

Summed attention weights are given by $\mathbf{w} \in \mathbb{R}^{N_k}$:

$$\mathbf{w}^T = \mathbf{1}_{N_q}^T \exp \left(\mathbf{A} - \lambda \mathbf{1}_{N_k}^T \right).$$

Define

$$\begin{aligned} \tilde{\mathbf{F}}_{r,s} &= \exp \left(\mathbf{A}_{I(r), J(s)} - \lambda_{r,s} \mathbf{1}_{|J(s)|}^T \right), \\ \mathbf{F}_{r,s} &= (\text{diag} \exp (\lambda_{r,s} - \lambda_r)) \tilde{\mathbf{F}}_{r,s} = \exp \left(\mathbf{A}_{I(r), J(s)} - \lambda_r \mathbf{1}_{|J(s)|}^T \right). \end{aligned}$$

If

$$\mathbf{w}_{r,s}^T = \mathbf{1}_{|I(r)|}^T \exp \left(\mathbf{A}_{I(r), J(s)} - \lambda_r \mathbf{1}_{|J(s)|}^T \right) = \mathbf{1}_{|I(r)|}^T \mathbf{F}_{r,s},$$

then $\mathbf{w} = [\sum_r \mathbf{w}_{r,s}]$. We can compute $\mathbf{w}$ and $\mathbf{Y}$ with an outer loop over r , inner loop over s . Initialize $\mathbf{w} = \mathbf{0}_{N_k}$. The iteration r works as follows:

- Map: Compute $[\lambda_{r,s}], [\tilde{\mathbf{F}}_{r,s}], [\mathbf{Y}_{r,s}]$ in parallel.
- Reduce: $(\lambda_r, \mathbf{Y}_r) = \text{reduce} ([\lambda_{r,s}], [\mathbf{Y}_{r,s}])$.
- Compute $\mathbf{w}_{r,s}^T = \mathbf{1}_{|I(r)|}^T \mathbf{F}_{r,s} = \exp(\lambda_{r,s} - \lambda_r)^T \tilde{\mathbf{F}}_{r,s}$. Add $\mathbf{w}_r = [\mathbf{w}_{r,s}]$ to $\mathbf{w}$.

Compared to standard FlashAttention, we need to first reduce along s in order to obtain λ_r , keeping $\tilde{\mathbf{F}}_{r,s}$ and $\lambda_{r,s}$ around.

A.6.2 Summed Attention Weights with FlexAttention

FlexAttention [13] stands out among fast SDPA codes by allowing the user to configure the computation in several ways (see also <https://pytorch.org/blog/flexattention/>). Here we describe how to compute summed attention weights $\mathbf{w}$ alongside the attention output $\mathbf{Y}$, by calling FlexAttention twice.

Recall that SDPA computes

$$\mathbf{Y} = \exp(\mathbf{A} - \lambda \mathbf{1}^T) \mathbf{V}.$$

The summed attention weights are

$$\mathbf{w} = \exp(\mathbf{A} - \lambda \mathbf{1}^T)^T \mathbf{1} = \exp(\mathbf{A}^T - \lambda \mathbf{1}^T) \mathbf{1} = \exp(\mathbf{A}^T) \tilde{\mathbf{v}}, \quad \tilde{\mathbf{v}} := \exp(-\lambda).$$

Up to softmax normalization, we can obtain this by calling a variant of SDPA again, flipping $\mathbf{Q}$ and $\mathbf{K}$, reverting the attention masking, and passing $\exp(-\lambda)$ as values. Importantly, FlexAttention returns λ with the option `return_aux = AuxRequest(lse=True)`. Now, if $\tilde{\lambda}$ denotes `lse` for the second call (with $\mathbf{Q}$ and $\mathbf{K}$ flipped), then:

$$\mathbf{w} = \exp(\mathbf{A}^T - \tilde{\lambda} \mathbf{1}^T + \tilde{\lambda} \mathbf{1}^T) \tilde{\mathbf{v}} = \left(\text{diag} \exp(\tilde{\lambda})\right) \exp(\mathbf{A}^T - \tilde{\lambda} \mathbf{1}^T) \tilde{\mathbf{v}} = \exp(\tilde{\lambda}) \circ \tilde{\mathbf{y}}.$$

Finally, trying to minimize numerical errors (we are using 16 bit data types), we use $\exp(-(\lambda - \bar{\lambda}))$ and $\exp(\tilde{\lambda} - \bar{\lambda})$, where $\bar{\lambda} = N_q^{-1} \mathbf{1}^T \lambda$ is the mean of λ . All in all:

- $(\mathbf{Y}, \lambda) = \text{SDPA}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$, $\bar{\lambda} = N_q^{-1} \mathbf{1}^T \lambda$.
- $(\tilde{\mathbf{y}}, \tilde{\lambda}) = \text{SDPA_rev}(\mathbf{K}, \mathbf{Q}, \exp(-(\lambda - \bar{\lambda})))$, then $\mathbf{w} = \exp(\tilde{\lambda} - \bar{\lambda}) \circ \tilde{\mathbf{y}}$.

Here, `SDPA_rev` differs from `SDPA` by the attention masking being reversed. FlexAttention allows to specify the attention mask as `block_mask(b, h, q_idx, kv_idx)`. The mask for `SDPA_rev` is given by flipping `q_idx` and `kv_idx` in the code for `SDPA`. Note that FlexAttention supports $\mathbf{V}$ to have a different (final) embedding dimension than $\mathbf{Q}, \mathbf{K}$. Maybe this even translates in the second call being faster than the first. All in all, compared to a single FlexAttention call and no attention weights, this is at most twice as expensive.

A.7 Sparse Attention and SotA Inference Libraries

In Section 3.3, we discuss the (somewhat surprising) fact that as of today, sparse attention is not much used in real-world practice, because existing implementations are too slow to be competitive with the state of the art. While a part of the latency gap between sparse attention and sequence or context parallelism is probably inherent, we argued in Section 3.3.1 some shortcomings of current sparse attention implementations are easy to eliminate by minor extensions of fast SDPA kernel codes.

Here, we comment on why sparse attention policies, such as H2O, are not supported in vLLM [30], the leading fast inference library. Details are found in <https://github.com/vllm-project/vllm/issues/10646>, <https://github.com/vllm-project/vllm/issues/12254>, <https://github.com/vllm-project/vllm/issues/5751>. In vLLM, KV caches are maintained as set of fixed-sized pages (or blocks). The main issue is that they require a page

to store KV content *across all heads*: KV information for a token is stored in the cache for all heads or for none. The cited RFCs mention that it would require significant changes to the memory layout and block manager abstractions to change that. However, for modern sparse attention (such as H2O), policies $\pi(b, h, t)$ depend on (h, t) in general: they select different tokens per head.

As detailed in Section 3.1, our implementation has no problems with this. We simply maintain dense buffers of shape (B, H_k, N_C, d_h) , where the cache length N_C is fixed independent of context width, and then use `torch.gather` and `torch.scatter` for read and write access. Whereas PagedAttention requires specific SDPA kernels, we can use existing dense SDPA codes, as long as we cater for causal masking (see Section 3.3.1). While not supported in our current implementation, we could build up KV cache buffers in chunks to cater for sequence lengths shorter than N_C , thereby solving the issue of unnecessary pre-allocations [30]. Finally, while `torch.gather` and `torch.scatter` access the buffer in a non-contiguous way, this is very subdominant to SDPA computations in our experience. In fact, when calling `scatter(keys, index, key_new)`, the final axis of `index` is always constant (so that the final buffer axis of size d_h is accessed contiguously), and optimized `scatter` and `gather` kernels could easily be implemented for this case if the PyTorch implementations do not already cater for this special case. One advantage of PagedAttention over our approach is that they can in principle represent different numbers of tokens per head or batch dimension, which can render sparse attention a bit more flexible. However, since vLLM requires each page to extend over all heads, this extra flexibility is not supported there.

As long as highly optimized and widely used inference libraries do not support sparse attention, it may remain underused. We hope that our work sparks some renewed interest in this direction.

A.8 Details on Related Work

Here, we provide additional details about relations of our method with prior work. OOMB [33] shares properties with our work, such as chunk-level processing, activation checkpointing, and efforts to compress KV cache buffers for *autograd*. Details on the relationship are as follows:

- Their implementation is better suited for representing KV caches exactly (no selection or compression) than ours. They implement a paged memory management like [30], which we do not (but see Section A.7 and comments in Section 3.1). However, despite all efforts in CPU offloading and activation checkpointing, they run into the same barrier as [34], in that the factor for the final chunk depends on *all* KV cache buffers of all layers, so cannot be represented by *autograd* on a single device. At this point, RingAttention [38] is the method of choice, and it is not clear why their library would improve on implementations such as MS-SWIFT [78].
- They deal with activation memory for GPU by activation recomputation, while we use activation checkpointing. In the former, activations are recomputed during the backward pass *from the start*, whereas in the latter, recomputation starts from the most recent checkpoint. The former is too slow to be useful, so our guess is their code actually uses activation checkpointing.

- The most important difference is how they deal with KV cache buffers as nodes in the *autograd* graphs, and what this implies for generality. This is also the biggest challenge we face, and we deal with it by a combination of nested checkpointing, delta encoding of KV cache buffers, and integration into *PyTorch* by way of autograd saved tensor hooks (Section A.4.3). Together with recording and replaying KV cache decisions, this *renders our implementation fully agnostic to the KV cache policy*: it works with any selection or compression policy (see Section 2 for many references). In contrast, they try to hide all nodes representing KV cache content from *autograd* altogether, so that none of this information can be placed in the computation graph. This is possible only by implementing a number of complex CUDA kernels, in which all inner derivatives w.r.t. these “KV cache nodes” are made explicit. Apart from substantial derivation and implementation complexity, their approach *must be specialized to the KV cache policy being used*. In fact, their paper only provides results for two specific sparse attention policies (LSA and DSA). Moreover, their paper is sparse on details how the hiding of KV cache buffers from *autograd* works in practice, since KV cache updates are tightly coupled with SDPA calls. In the end, their implementation may not be agnostic to SDPA kernels, which given the speed of development of SDPA would be a major drawback.
- Both their and our implementation make use of CPU offloading of activations, KV cache buffers, and head gradients. They claim to have done this asynchronously, as in [71], which hides latency. We have also experimented with this, but did not so far achieve significant speedups. Moreover, asynchronous transfer requires double buffering, which drives up GPU memory requirements. Still, more effort in this direction is warranted.

A.9 Open Source Library KeysAndValues. Experiments

For all experiments above, fine-tuning with our method (column *us* in Table 1) and inference with sparse attention (all policies) were done with a new open source library for long context fine-tuning and inference: **KeysAndValues** (https://github.com/aws-labs/keys_values).

Apart from efficient code for our fine-tuning method, the library provides clean and simple abstractions for sparse attention and key-value caches of limited size. Among its features are:

- Long context fine-tuning on a single GPU (this work).
- Several variants of the H2O KV cache policy [72]. The library provides a generic implementation for any cache logic of the form $\pi(b, h, t)$ which makes use of summed attention weights.
- Quantization of KV cache buffers.
- Integration of FlexAttention [13], FlashInfer [68], FlashAttention [9, 48] and eager SDPA behind a common multi-head self-attention interface. This includes summed attention weights (for H2O-like policies), as well as a proper **backward** implementation.
- Model implementations and inference code is from LitGPT (<https://github.com/lightning-ai/litgpt>), which allows for almost any Hugging Face checkpoint to be

used. However, while bringing modern KV caching to Hugging Face would require hacking several code files *separately for every single model*, you can apply your KV cache policy or attention approximation to almost all models with few changes of common code.

- Support of CPU offloading of KV cache buffers and model weights.
- Support of distributed training (distributed data parallel, CPU offloading of model weights optional). Support of distributed evaluation.

With this library, we do not intend to compete with vLLM [30] or SGLang [79], which include more low level optimizations and support of latest GPU architectures. Instead, we make it easy for researchers to explore new KV cache policies, post-time training ideas, or unusual multi-head self-attention approximations, providing clean abstractions of these concepts which can be used and extended without having to deal with intricate implementation details of existing high-performance libraries.

A.9.1 Running Our Experiments

Once `KeysAndValues` has been properly installed, the training runs for our method can be reproduced as follows. You need to be on an instance with at least four Nvidia A100 GPUs with 40 GB of RAM. We used AWS EC2 `p4d.24xlarge` instances, which have 8 A100 GPUs, running two experiments in parallel on each instance.

```
export DATASET_KEY="nq"; \
export DATASET_SIZE="128k"; \
export POLICY_NAME="h2o-orig"; \
export CACHE_LENGTH="32768"; \
export CHUNK_SIZE="2048"; \
export EVAL_STEPS=10; \
CUDA_VISIBLE_DEVICES="0,1,2,3" \
PYTORCH_ALLOC_CONF=expandable_segments:True \
KEYSVALS_LOG_DIR="./finetune/helmet.${DATASET_KEY}_${DATASET_SIZE}/${POLICY_NAME}_cs${CHUNK_SIZE}/logs" \
python3 keys_values/finetune.py finetune-long-lora \
  Qwen/Qwen3-4B-Instruct-2507 \
  --out_dir ./finetune/helmet.${DATASET_KEY}_${DATASET_SIZE}/${POLICY_NAME}_cs${CHUNK_SIZE} \
  --precision bf16-true \
  --verbose some \
  --devices 4 \
  --data Helmet \
    --data.dataset_key ${DATASET_KEY} \
    --data.max_length ${DATASET_SIZE} \
    --data.metadata_dir ./data \
    --data.trainloader_longest_first True \
  --train.save_interval ${EVAL_STEPS} \
  --train.micro_batch_size 2 \
  --train.epochs 5 \
  --train.average_loss_per_batch True \
  --eval.interval ${EVAL_STEPS} \
  --eval.initial_validation True \
  --eval.use_sample_metric False \
  --kv_cache.cache_length ${CACHE_LENGTH} \
  --kv_cache.chunk_size ${CHUNK_SIZE} \
  --kv_cache.name ${POLICY_NAME}-torch-quantized8 \
  --grad.layers_per_cell 1 \
  --grad.layercp.qname default \
  --grad.cachecp.qname torch-quantized8 \
  --grad.chunks_per_cell_multiplier 1 \
  --optimizer.name AdamW \
  --optimizer.learning_rate 0.0005
```

Once all desired training runs have finished, evaluations (on the test sets) can be run as follows.

```
CUDA_VISIBLE_DEVICES="0,1,2,3" \
PYTORCH_ALLOC_CONF=expandable_segments:True \
KEYSVALS_LOG_DIR="./finetune/evaluation/myruns/logs" \
python3 keys_values/finetune.py eval-long-ext \
  ./myruns.yaml \
```

```
—verbose some \
—devices 4 \
—batch_size 2 \
—use_sample_metric True \
—sample_metric_max_generated_tokens 20 \
—num_store_generated_samples 1000
```

Here, `myruns.yaml` is a YAML file containing entries of this form:

```
— out_dir: ./finetune/helmet_nq.64k/h2o_cs2048
  model_type: lora
  eval_tasks:
    — step-000420
```

For each setup, evaluations can be run for different checkpoints `step-000***` stored alongside training. In our experiments, for each setup, we select the checkpoint which minimize validation loss. We refer to `README.md` for further details on how to aggregate evaluation results and create result tables.